

Fog Carport Rapport

Klasse A - Gruppe O
03-06-2026



Fabian Leonhard Strøm - E-mail: cph-fs180@stud.ek.dk - Github: Fabian-str
Mikkel Fuglsig Nielsen - E-mail: mini0011@stud.ek.dk - Github: MikkelMattern
Nali Ahmadi - E-mail: cph-na282@stud.ek.dk - Github: Nali1234-lab

Indholdsfortegnelse

Indholdsfortegnelse.....	1
Links.....	2
Github.....	2
Website.....	2
Admin bruger.....	2
Video.....	2
Indledning.....	2
Baggrund.....	3
Virksomheden / Forretningsforståelse.....	4
Interessentanalyse.....	6
Teknologivalg.....	8
Krav.....	9
Arbejdsgang der skal IT-støttes.....	9
User stories.....	12
Domæne model & ER-diagram.....	14
Domæne model.....	14
ER-diagram.....	16
Navigationsdiagram & Mockups.....	20
Valg af arkitektur.....	22
Packages og struktur.....	22
Design patterns.....	24
Særlige forhold.....	26
Udvalgte kodeeksempler.....	29
OrderObserver – Observer Pattern.....	29
MaterialService – Factory og Template Method Pattern.....	30
Status på implementering.....	32
Implementerede funktioner.....	32
Kendte mangler.....	33
Kvalitetssikring (test).....	34
Autoriserede tests.....	34
Unit Test.....	36
User Acceptance tests.....	38
Proces.....	39
Arbejdsprocessen Faktuelt.....	39
Arbejdsprocessen Reflekteret.....	40
Bilag.....	42
User stories.....	42
User story estimat.....	46
Klassediagram.....	47
Kanban Board.....	48
Kode eksempler.....	49

Links

Github

Repository - <https://github.com/CarportProject/CarportProject>

Fabian - <https://github.com/Fabian-str>

Mikkel - <https://github.com/Mikkelmattern>

Nali - <https://github.com/Nali1234-lab>

Website

<https://carport.mikkelmattern.dk/>

Admin bruger:

Email: admin@example.com

Password: ADMIN

Det er også muligt at indtaste konami koden (↑,↑,↓,↓,←,→,←,→, b, a) for at blive logget ind som admin.

Video

<https://youtu.be/R0WDRF6-Ze4>

Indledning

Denne rapport henvender sig til fagfæller med et fagligt niveau svarende til andet semester på datamatikeruddannelsen. Formålet med rapporten er at dokumentere udviklingen af en webapplikation til XL-BYG Fog, tidligere kendt som Fog.

XL-BYG Fog håndterer i dag salg og administration af skræddersyede carporte gennem en proces, der kræver en betydelig mængde manuelt arbejde. Projektets formål er derfor at udvikle en webapplikation, som kan understøtte og automatisere dele af denne proces.

Systemet er udviklet i samarbejde med XL-BYG Fog, som har fungeret som kunde og stillet en række krav til systemets funktionalitet. Teamet bag projektet består af tre udviklere med forskellige faglige styrker, som har arbejdet sammen om at omsætte kundens krav til user stories, design, database, backend-logik, frontend og test.

Baggrund

Virksomheden XL-BYG Fog, tidligere kendt som Fog og fremadrettet omtalt som Fog, er et trælast- og byggecenter fordelt på ni lokationer i Danmark. Fog sælger et bredt udvalg af værktøj, byggematerialer og specialløsninger, herunder skræddersyede carporte.

Salget af skræddersyede carporte kræver i dag en betydelig mængde manuelt arbejde fra Fogs side. Processen indebærer blandt andet, at sælgere manuelt skal oprette tilbud, indtaste kundens oplysninger, beregne priser og oprette ordrer efter gennemført betaling. Formålet med dette projekt er derfor at udvikle en webapplikation, som kan automatisere store dele af dette workflow og samtidig skabe en mere effektiv og brugervenlig løsning for både kunder og sælgere.

Systemets primære funktion er at give kunder mulighed for at udfylde en forespørgsel på en skræddersyet carport. Dette hænger især sammen med *User Story 4*, hvor kunden skal kunne vælge carport type og udfylde en formular med sine ønskede specifikationer. Kunden skal blandt andet kunne vælge carportens længde, bredde, tagmateriale og eventuelt tilføje et redskabsskur med ønskede dimensioner. Kunden skal desuden kunne vælge mellem forskellige tagtyper, herunder fladt tag eller tag med høj rejsning, afhængigt af behov og præferencer. Når forespørgslen er udfyldt, skal den automatisk registreres i systemet, og der skal sendes en mail til både kunden og sælgeren.

Derudover skal beregning af tilbud og håndtering af forespørgsels-data automatiseres. Det betyder, at systemet skal kunne behandle kundens indtastede oplysninger, beregne relevante materialer, udregne en samlet pris og generere en tegning af den ønskede carport. Disse dele understøtter især det forbedrede flow fra *User Story 4*, da kundens forespørgsel danner grundlag for den videre behandling af ordren.

Et vigtigt forretningsmæssigt krav er, at styklister først gøres tilgængelige efter gennemført betaling. Dette skal beskytte Fog mod, at kunder anvender virksomhedens arbejdstegninger og styklister til at få opført carporte billigere hos konkurrerende virksomheder. Dette krav hænger især sammen med *User Story 10*, hvor kunden efter betaling skal modtage en kvittering og få adgang til styklisten.

Fra sælgerens perspektiv skal systemet give mulighed for at se og behandle indkomne forespørgsler. Sælgeren skal kunne få et overblik over alle forespørgsler, se detaljer om kundens ønskede carport, vurdere den beregnede pris og acceptere eller afvise forespørgsler. Sælgeren skal desuden kunne justere tilbudspriser og redigere materialeoplysninger, så priserne i systemet kan tilpasses Fogs aktuelle behov. Disse funktioner hænger blandt andet sammen med *User Story 3, 6, 7 og 8*.

Virkksomheden / Forretningsforståelse

Risk ID	Risiko	Alvor	Sandsynligt	Risiko-niveau	Plan for forebyggelse/afværgning
1	Langtidssygdom på teamet	Tolereres	Muligt	Høj	Omfordele arbejdsfordelingen.
2	Manglende kompetencer i teamet	Acceptabel	Muligt	Medium	Få kompetencer, eller finde en anden måde at opnå samme resultat.
3	Martin fraværende	Uønsket	Usandsynlig	Høj	Planlægge at have mindst 2 nødkontakter hos Fog, som kan erstatte Martin midlertidigt.
4	Team-medlem fraværende	Tolereres	Muligt	Lav	Tale med gruppemedlem om hvorfor de har været fraværende eller Omfordele arbejdsfordelingen.
5	Fog går konkurs	Acceptabel	Usandsynlig	Ekstrem	Tale med projektleder.
6	Forsinkelser i tidsplanen	Uønsket	Muligt	Medium	Crunch time.
7	Misforståelse mellem kunder og udviklere	Uønsket	Sandsynligt	Medium	Kontakt Martin eller nødkontakter, for at løse misforståelsen.
8	Dårlig kommunikation i teamet	Tolereres	Muligt	Lav	Forbedre kommunikation.
9	Afhængighed af tredjepart	Tolereres	Muligt	Høj	Have fallback løsninger for hver afhængighed.
10	Slutproduktet opfylder ikke kravene	Tolereres	Sandsynligt	Ekstrem	Find ud af hvad der mangles og implementer det i slutproduktet.
11	Utilstrækkelig testning	Tolereres	Muligt	Høj	Tilføj flere tests.
12	Konflikt i gruppen/uenighed	Tolereres	Sandsynligt	Lav	Ved uenighed: Foreslå 2-3 løsninger – stem om dem. Løs konflikten.
13	Uklare kravspecifikationer	Uønsket	Sandsynligt	Høj	Gå ikke videre med kodning, før I har fået svar på det uklare.
14	Ændringer i projektets scope	Uønsket	Muligt	Høj	Kontakt projektleder
15	Ulige arbejdsfordeling på teamet	Uønsket	Muligt	Lav	I stedet for én stor opgave, del opgaverne op i små bidder.
16	Beregningsalgoritmen for carporte giver forkerte resultater	Tolereres	Muligt	Ekstrem	Fiks koden.
17	Styklisten genereres med forkerte materialer eller mængder	Uacceptabelt	Muligt	Medium	Fiks koden.
18	Manglende betaling fra Fog	Uacceptabelt	Usandsynlig	Ekstrem	Teamet beholder det stykke arbejde de har lavet, indtil betaling er gennemført. Eventuelt sker betaling før arbejdet fra teamets side går i gang.
19	Moms udregning giver uønskede resultater	Tolereres	Muligt	Ekstrem	Fiks algoritmen.
20	Kundedata lækkes	Uacceptabelt	Usandsynlig	Ekstrem	Krypter kundedata, implementer adgangskontrol.

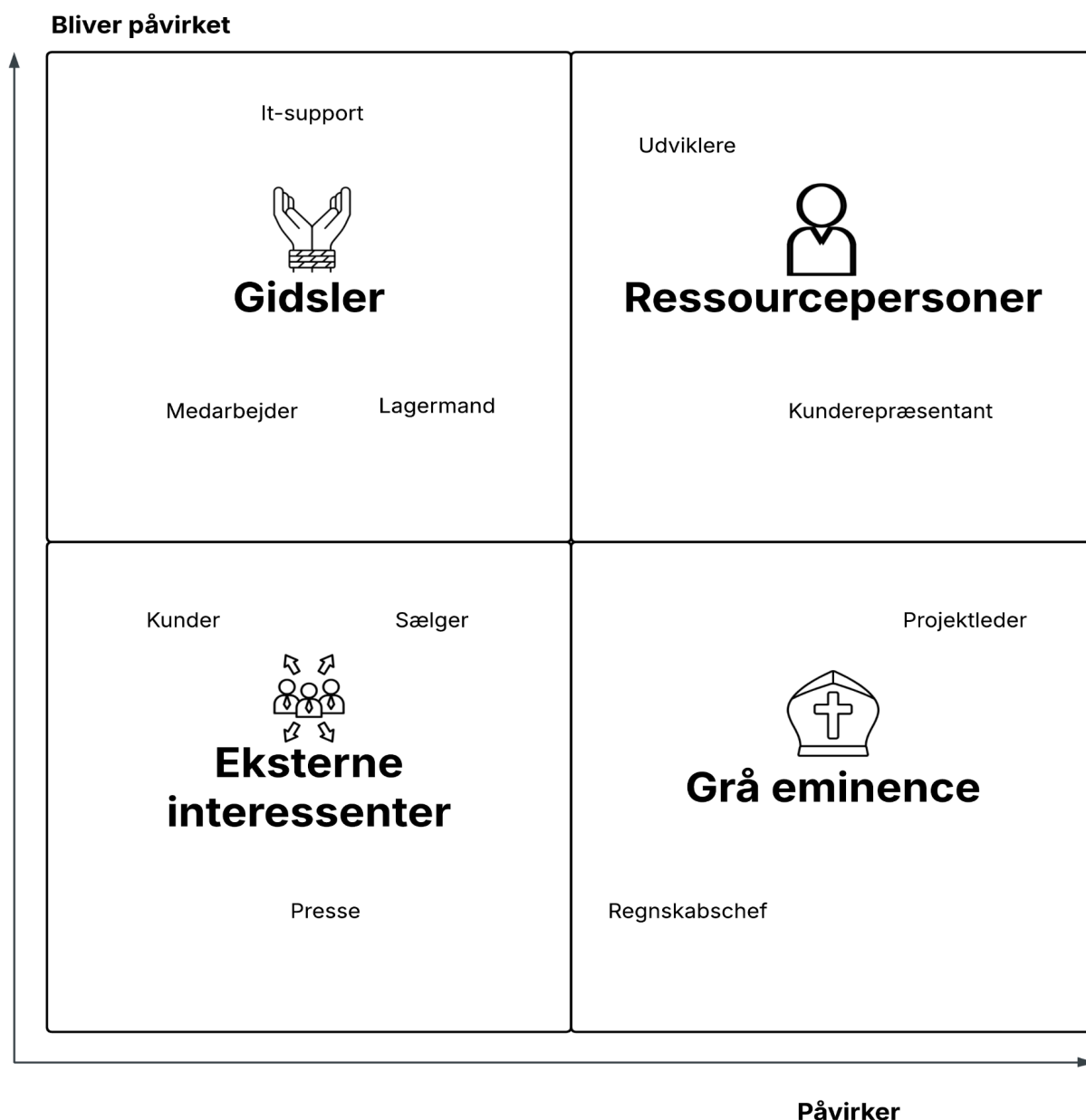
For at få en bedre forståelse af Fogs behov og projektets rammer udarbejdede vi både en risikoanalyse og en interessentanalyse. Formålet var at få overblik over, hvilke personer og grupper der kunne påvirke projektet, samt hvilke risici der kunne få betydning for både udviklingsprocessen og det færdige system.

Risikoanalysen blev brugt til at identificere potentielle problemer tidligt i projektforsløbet. På den måde kunne vi forholde os til risiciene, før de opstod, og overveje hvordan de kunne forebygges eller håndteres. To af de mest kritiske risici var læk af kundedata og langtidssygdom på teamet.

Risikoen for læk af kundedata blev vurderet som uacceptabel, fordi systemet håndterer personoplysninger som navn, adresse, telefonnummer og e-mail. Dette havde betydning for vores fokus på adgangskontrol og sikkerhed, særligt i forbindelse med sælgerens adgang til kundedata og forespørgsler. Dette hænger blandt andet sammen med *User Story 6* og *User Story 7*, hvor sælgeren skal kunne se forespørgsler, detaljer og kundeoplysninger. Derfor var det vigtigt, at disse dele af systemet kun kunne tilgås af brugere med de rette rettigheder.

Langtidssygdom på teamet blev vurderet som en risiko, der kunne tolereres, men som stadig kunne få stor betydning for projektets fremdrift. Vores plan var at omfordele arbejdsopgaverne, hvis et teammedlem blev fraværende i længere tid. Dette var særligt vigtigt, fordi flere user stories bestod af sammenhængende dele. For eksempel krævede *User Story 4* både formular, validering, databasegemning, mailafsendelse og PDF-tegning. Hvis én person blev forhindret i at arbejde videre, skulle resten af teamet derfor kunne overtage eller færdiggøre dele af arbejdet.

Interessentanalyse



Analysen har til formål at give et overblik over, hvem der påvirker projektet og hvem der bliver berørt af det.

IT-support er placeret som gidsler, da de leverer den tekniske infrastruktur projektet bygger på. De har ingen direkte beslutningsmagt, men er en kritisk afhængighed efter overlevering, da de skal vedligeholde og supportere systemet i den daglige drift. Vælger vi teknologi de ikke forstår eller kan håndtere, risikerer vi et system der ikke kan holdes kørende i praksis. Hvis IT-support inddrages aktivt i udviklingsfasen som repræsentanter, kan de omklassificeres til ressourcepersoner.

Kunderepræsentanten har værdifuld viden om kundernes behov og fungerer som bro mellem projektet og slutbrugerne. Selv uden formel beslutningsmagt er input kritisk for at udvikle et produkt der lever op til kundens forventning. Ignorerer kunderepræsentanten, opstår der misforståelser mellem kunder og udviklere, hvilket kan føre til fejl og dyre ændringer. Dette er det mest kritiske forhold at vedligeholde, da det er dem der repræsenterer kunden.+

Pressen er placeret som ekstern interessant. De har ikke noget formel rolle eller beslutningsmagt i vores projekt. Rollen på som et eksempel på nogen der kunne få større indflydelse i tilfælde af u hensigtsmæssige ændringer i vores projekt, som datalæk.

Projektlederen er placeret som ressourceperson, da de besidder formel magt gennem deres rolle og har indsigt i tidsplaner, ressourcer og aftaler. De påvirker direkte projektets fremdrift og udviklernes arbejde. Positionen kan ændre sig hvis ledelsen over dem bliver mere involveret og tilsidesætter deres beslutninger, i så fald kan projektlederen omklassificeres lavere, dog stadig i ressourceperson, da deres reelle indflydelse vil være reduceret til koordinerende opgaver.

Kunder er de endelige brugere af produktet, og projektet er lavet for at forbedre deres oplevelse. Vi kan ikke overse dem, for utilfredse kunder betyder et fejlet projekt. Kunder kan rykke sig, som hvis de inddrages i fokusgrupper, vil de blive ressourcepersoner.

Dog er det begrænset, hvad vi har kunne bruge analysen til, da vi ikke har kunne komme ud og interagere med de forskellige roller, da vores projekt er et skoleprojekt.

Teknologivalg

Programmeringssprog

Java 21

Backend framework

Javalin 6.7.0

Templating

Thymeleaf 3.1.3

Database

PostgreSQL (hostet på DigitalOcean Droplet)

HikariCP 6.2.1
(connection pool)

PostgreSQL JDBC
driver 42.7.11

Sikkerhed

jBCrypt 0.4 (password
hashing)

Email

Jakarta Mail API 2.1.3

Angus Mail 2.0.4

Test

JUnit Jupiter 5.12.0

Byggeværktøj

Maven (med Maven
Shade Plugin 3.4.1 til
deployment)

Versionsstyring

Git / GitHub (med
branch protection og
pull requests)

Hosting

DigitalOcean Droplet
(Ubuntu)

Udviklingsmiljø

IntelliJ IDEA

Containerisering:

Docker

Docker Compose
(PostgreSQL database)

Øvrige biblioteker

Jackson 2.18.3 (JSON)

SLF4J 2.0.17 (logging)

dotenv-java
(miljøvariabler til tests)

Apache PDFBox 3.0.1
(PDF merge)

Flying Saucer PDF
9.1.22 (HTML/tabel →
PDF)

iText 2.1.7 (bruges af
Flying Saucer)

Apache Batik 1.17
(SVG → PDF)

Apache FOP 2.9
(bruges af Batik)

Krav

Arbejdsgang der skal IT-støttes

AS-IS aktivitetsdiagram

AS-IS-diagrammet har tre svømmebaner: Kunde, Sælger og Logistik.

Kunden vælger carport-type på hjemmesiden og udfylder et ordreformular.

Sælger modtager ordren, udregner et tilbud manuelt, ringer til kunden og venter på kundens svar.

Hvis kunden siger ja, sender sælger en bekræftelse manuelt. Hvis kunden siger nej, sender sælger en afvisning.

Betalingslinket sendes også manuelt af sælger. Når kunden har betalt, opretter sælger selv ordren og sender den videre til logistik – alt sammen manuelt.

Logistik modtager ordren, laver en PDF, en tegning og en stykliste, og håndterer derefter pakning og forsendelse.

Kernen i AS-IS er, at kunden godt nok starter digitalt på hjemmesiden, men alle trin bagefter udføres i hånden af sælger og logistik.

Sælger skal blandt andet vente på kundens ja eller nej, før han kan handle videre.

TO-BE aktivitetsdiagram

I TO-BE-Diagrammet vælger kunden carport-type og udfylder en formular på hjemmesiden.

Systemet beregner herefter automatisk et tilbud ud – og laver allerede her en PDF, en tegning og en stykliste som tilbudsdokumenter.

Sælger skal kun gøre én ting: han skal sige ja eller nej til systemets beregning. Hvis sælger siger ja, sendes et betalingslink automatisk til kunden. Hvis sælger siger nej, sendes der automatisk en afvisningsmail.

Når kunden har betalt, opretter systemet selv ordren og sender den automatisk videre til logistik.

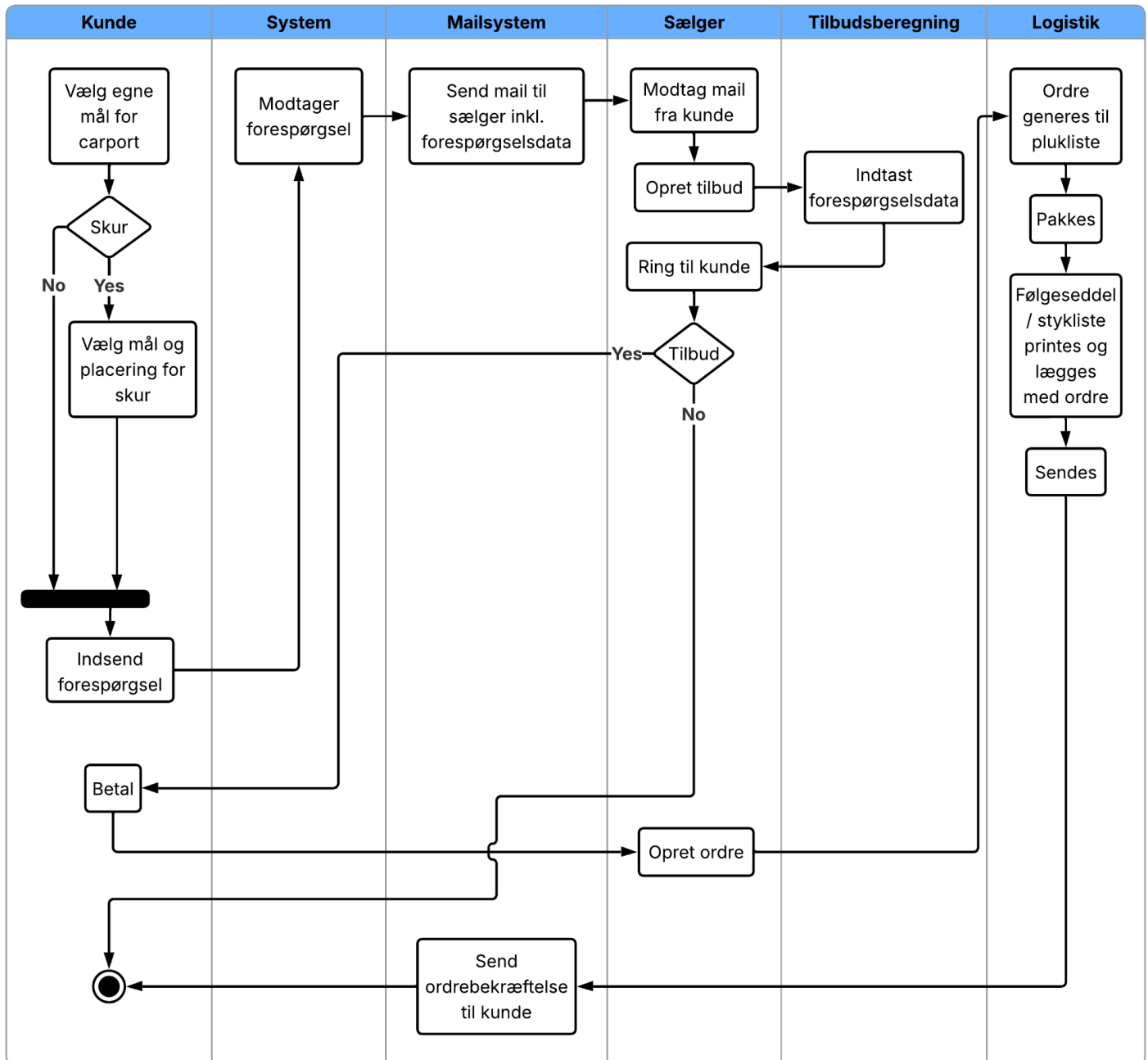
Logistik modtager ordren uden nogen manuel videresendelse. Til sidst pakkes og afsendes carporten.

Den store forskel fra AS-IS er, at sælger i dag skal gøre næsten alt selv: beregne, ringe, vente på kundens svar, sende mails, oprette ordre og sende videre til logistik.

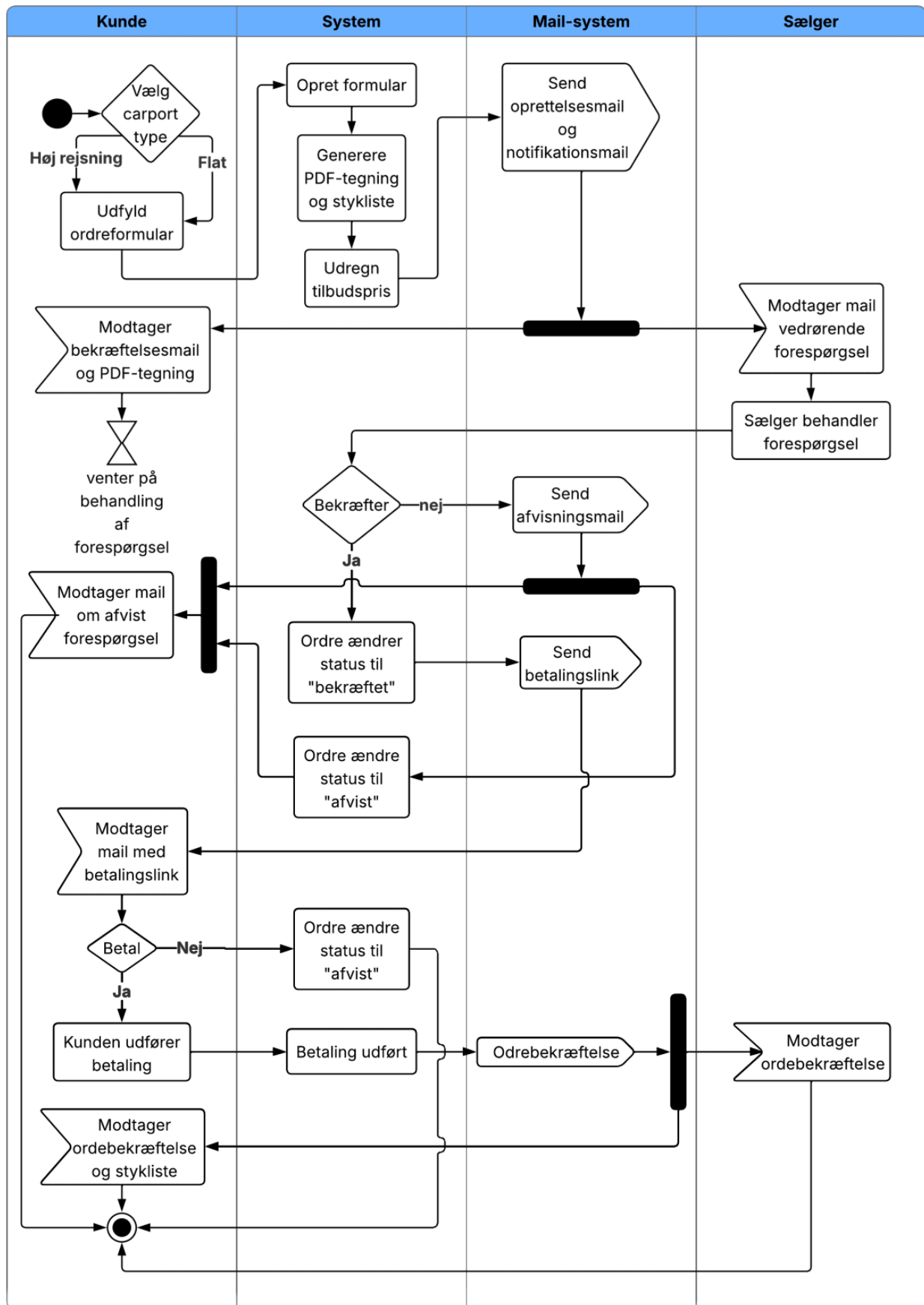
I den fremtidige løsning klarer systemet det hele. Sælger skal kun sige ja eller nej – han venter ikke på kunden, fordi systemet allerede har beregnet alt.

Logistik får ordren automatisk uden at vente på, at sælger sender den manuelt.

AS-IS aktivitetsdiagram (Figur)



TO-BE aktivitetsdiagram (Figur)



User stories

Den fulde samling af user stories findes i bilaget. I dette afsnit fremhæves to centrale user stories, som viser, hvordan kravene fra Fog og arbejdsgangene i aktivitets diagrammerne er blevet omsat til konkrete funktioner i systemet.

En af de mest centrale user stories i projektet er *User Story 4*:

“Som kunde ønsker jeg at vælge carporttype og udfylde en formular med mine ønskede specifikationer, så jeg kan sende en forespørgsel på en carport.”

Denne user story er central, fordi den markerer starten på kundens flow i systemet. I AS-IS-diagrammet starter kunden også med at vælge carport type og udfylde en ordreformular, men efterfølgende foregår størstedelen af processen manuelt hos Fog. I TO-BE-diagrammet bliver denne arbejdsgang i stedet IT-understøttet, da systemet overtager flere af de opgaver, som tidligere blev udført manuelt.

Dette ses især i følgende acceptkriterium:

“Givet at formularen er korrekt udfyldt, når jeg sender forespørgslen, så gemmes forespørgslen i databasen, og en mail bliver sendt til en sælger og mig selv som kunde.”

Acceptkriteriet viser første skridt i det forbedrede workflow for Fog. I stedet for at en medarbejder manuelt skal indtaste og behandle forespørgslens specifikationer, bliver forespørgslen gemt direkte i databasen. Samtidig sendes der automatisk en mail til både sælger og kunde. Dette gør forespørgslen lettere tilgængelig og danner grundlag for den videre automatisering af ordreflowet.

User Story 4 blev estimeret som *Large*, fordi den består af flere sammenhængende dele. Den krævede blandt andet formularvisning, validering af brugerinput, håndtering af carporttype, oprettelse af forespørgsel i databasen samt afsendelse af mail og PDF-tegning. Derfor blev den også nedbrudt i flere mindre tasks, så de enkelte dele kunne implementeres og testes hver for sig.

I implementeringen kan *User Story 4* blandt andet spores til formularflowet i `UserController`, hvor routes til valg af carport type og indsendelse af formularen registreres. Her håndteres blandt andet `/carport/flat-roof`, `/carport/raised-roof` og `/carport/send-form`. Når formularen indsendes, valideres input gennem `OrderFormService`, hvorefter ordren oprettes gennem `OrderService`.


```
app.get("/carport/raised-roof", ctx -> getRaisedRoof(ctx, connectionPool));  
app.get("/carport/flat-roof", ctx -> getFlatRoof(ctx, connectionPool));  
app.post("/carport/send-form", ctx -> buildOrderWithForm(ctx, connectionPool));
```

Routes i UserController, som kobler valg af carporttype og indsendelse af formularen til User Story 4.

Den anden user story, vi har valgt at fremhæve, er *User Story 10*:

“Som kunde ønsker jeg en kvittering og stykliste, så jeg kan se, hvad jeg har købt.”

Denne user story er central, fordi den tager udgangspunkt i et vigtigt forretningskrav fra Fog. Kunden må først få adgang til styklisten efter gennemført betaling. Dette skyldes, at styklisten indeholder informationer, som kunden potentielt kunne bruge til at få bygget carporten billigere et andet sted. Derfor er det vigtigt for Fog, at styklisten først bliver gjort tilgængelig, når kunden reelt har accepteret tilbuddet og betalt.

Dette ses især i følgende acceptkriterium:

“Givet at jeg har betalt, når jeg tilgår systemet, så får jeg adgang til en stykliste.”

Acceptkriteriet viser tydeligt koblingen mellem betaling og adgang til styklisten. I TO-BE-diagrammet ses dette ved, at kunden først betaler, hvorefter systemet ændrer ordrestatus og sender ordrebekræftelse samt stykliste. På den måde bliver en manuel arbejdsgang fra AS-IS-processen erstattet af et mere automatiseret flow, hvor systemet selv håndterer statusændring, kvittering og adgang til dokumenterne.

User Story 10 blev estimeret som *Medium*, fordi den krævede flere sammenhængende dele, men ikke var lige så omfattende som selve ordreformular. Den krævede blandt andet håndtering af betaling og ordrestatus, afsendelse af kvitteringsmail, adgangskontrol til styklisten og visning af ordrestatus for sælgeren. Flere af disse dele byggede dog videre på funktionalitet, som allerede var blevet sat op i forbindelse med andre user stories, eksempelvis brugeroprettelse, mail funktionalitet og håndtering af ordredata. Derfor vurderede vi, at arbejdsomfanget var mindre end *User Story 4*, men stadig stort nok til et *Medium*-estimat. User story'en blev derfor nedbrudt i mindre tasks, så betaling, statusændring, mail og styklisteadgang kunne implementeres og testes hver for sig.

I implementeringen kan *User Story 10* spores til betalings- og mailflowet. Når kunden betaler, ændres ordrestatus til `PAID`. Denne statusændring håndteres efterfølgende i `CustomerEmailObserver`, hvor metoden `handlePaidStatus()` står for at generere og sende kundens PDF med stykliste og tegning.

```

case PAID -> {
    handlePaidStatus(order, customerName, orderId, connectionPool);
    return;
}

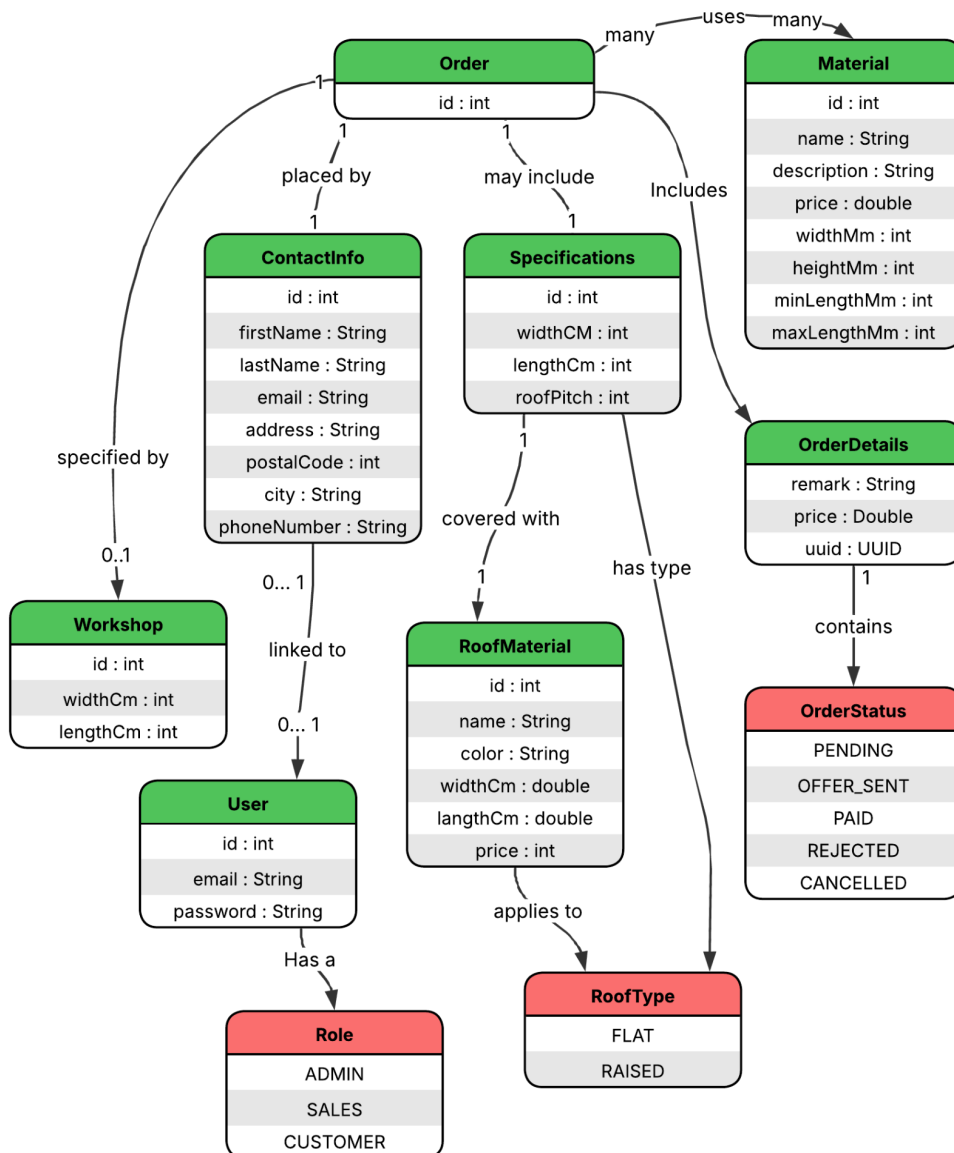
```

Udsnit fra *CustomerEmailObserver*, hvor status *PAID* udløser håndteringen af kvittering, stykliste og PDF til kunden.

Domæne model & ER-diagram

Domæne model

Domænemodellen beskriver de centrale koncepter i vores bestillingssystem. Kernen er en forespørgsel som en sælger behandler og sender tilbud på.



Domænemodellen har til formål at repræsentere verden som den er, da dette gør det nemmere at skrive kode og konceptualisere slutproduktet. Vi har taget udgangspunkt i `Order` som det centrale omdrejningspunkt i projektet.

Når en kunde opretter en ordre i systemet, ser de ikke den adskillelse som findes i vores domænemodel. Derimod udfylder de én samlet formular med alle felter i deres egen rækkefølge (User Story 4). Vi har dog valgt at opdele dataen på baggrund af hvordan den bliver brugt: `Specifications` bruges til at udregne prisen på carporten samt generere SVG-tegningen, mens `ContactInfo` bruges når vi skal sende mails til kunden.

Det er vigtigt at skelne mellem `User` og `ContactInfo`. En `User` er en bruger på siden med en rolle defineret i `Role`-enumet, og oprettes via login-flowet.

Rollen styrer hvilke sider brugeren kan tilgå. `ContactInfo` indeholder derimod de oplysninger, kunden har udfyldt i ordreformularen, såsom navn, adresse og telefonnummer.

Forholdet mellem dem er, at en `ContactInfo` har en tilknyttet `User`, hvilket giver mulighed for fremtidigt at sammenkoble brugerens konto med deres ordreoplysninger. Vi har dog ikke implementeret dette fuldt ud, da ordrefrekvensen per bruger forventes at være lav.

Vores **aggregate root** er `Order`, som ejer alle de andre klasser via komposition. Den indeholder `ContactInfo`, `Specifications`, `Workshop` og `OrderDetails`.

`Order` er den centrale klasse i projektet, da den samler al data og repræsenterer hele ordren.

`OrderDetails` er en record der indeholder ordrens metadata: status via `OrderStatus`-enumet, pris, et valgfrit notat samt et `UUID` der bruges til den unikke betalingsside.

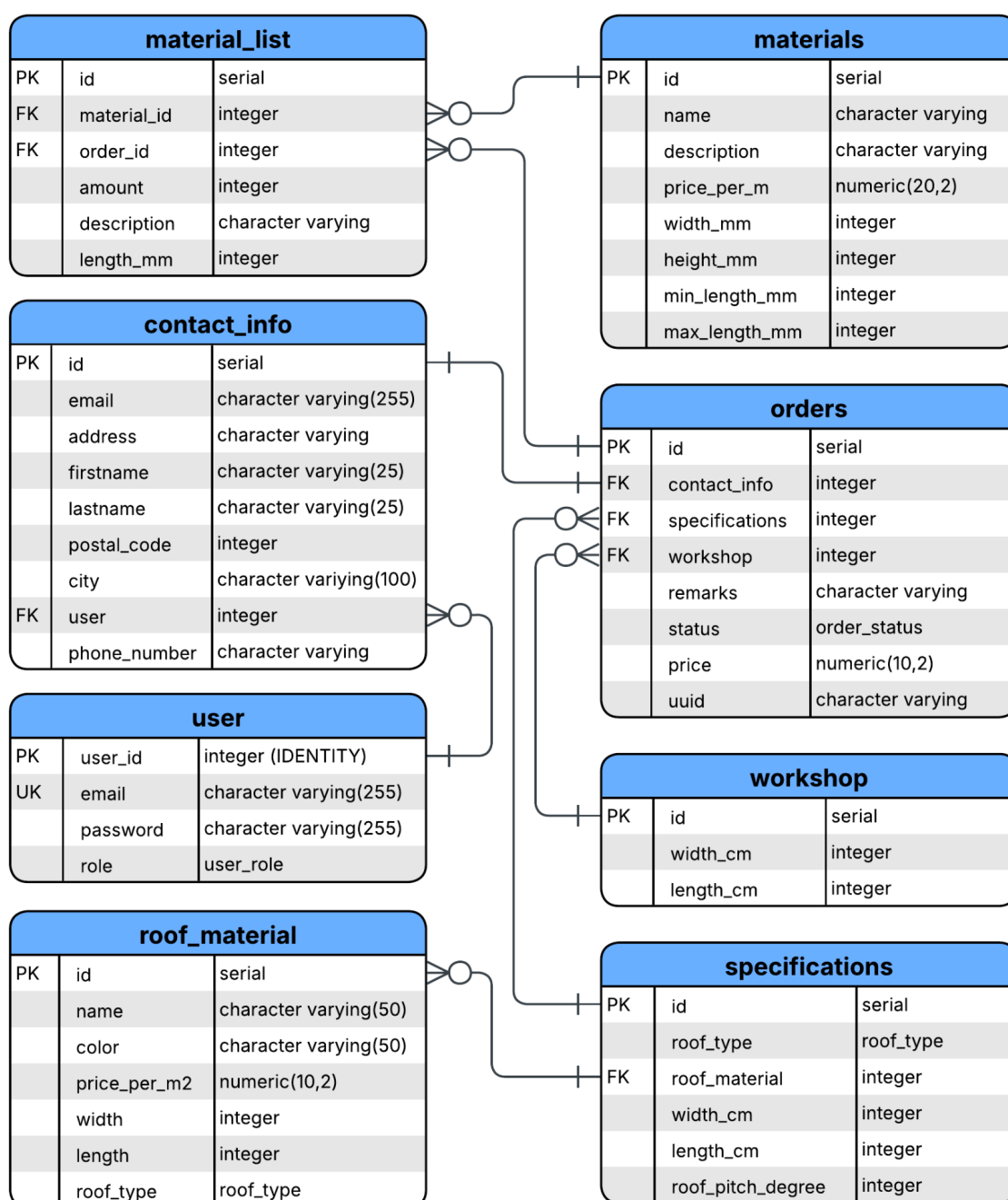
`Specifications` indeholder de tekniske mål på carporten samt tagtype via `RoofType`-enumet og den valgte tagbeklædning via `RoofMaterial`-klassen.

`Workshop` repræsenterer et eventuelt redskabsrum tilknyttet carporten. Den oprettes på baggrund af de felter i formularen der omhandler redskabsrummets dimensioner. Vi har valgt at adskille den fra `Specifications`, da domænemodellen skal afspejle koncepterne snarere end blot formularfelterne.

`Material` repræsenterer et element i styklisten der sendes til kunden efter betaling. `MaterialListEntry` er en record der kobler et `Material` til en specifik ordre med en mængde, længde og beskrivelse.

Vi er klar over at id-felterne ikke strengt taget hører til i en domænemodel, men vi har valgt at inkludere dem for at tydeliggøre relationerne til databasen. Samlet set afspejler domænemodellen de centrale koncepter i systemet og danner grundlag for det efterfølgende ER-diagram.

ER-diagram



En af de dele af projektet, som størstedelen af systemet var afhængigt af for at kunne udvikles, var databasen. Derfor var udviklingen af ER-diagrammet også noget af det første, vi påbegyndte i projektet. Her startede vi med at identificere nogle af de vigtigste elementer, som var nødvendige for at gøre webapplikationen funktionel. Nogle af disse elementer var blandt andet tabellerne `order`, `user` og `materials`, som repræsenterer centrale dele af systemets funktionalitet og de data, systemet arbejder med.

I databasen er der valgt automatisk genererede IDs som primary keys (PK) i alle tabeller. Dette blev gjort for at skabe simple og stabile relationer mellem tabellerne, samt gøre det muligt entydigt, at identificere hver enkelt række i databasen. En fordel ved denne løsning er, at relationerne ikke afhænger af data, som potentielt kan ændres senere, eksempelvis en e-mailadresse eller et navn. Samtidig gør det databasen mere overskuelig og effektiv at arbejde med, både i forhold til relationer, queries og vedligeholdelse af systemet.

I ER-diagrammet ses desuden en række foreign keys (FK), som er anvendt for at skabe relationer mellem tabellerne og sikre referentiel integritet. Dette betyder blandt andet, at en ordre ikke kan referere til en bruger, specifikation eller et materiale, som ikke eksisterer i databasen. På den måde hjælper relationerne med at opretholde databasens konsistens og reducere risikoen for fejl i systemet.

Ud over primary keys og foreign keys benyttes der også en unique key (UK) i databasen. Denne anvendes i tabellen "user", hvor email-attributten er defineret som unik. Det betyder, at der ikke kan oprettes flere brugere med den samme e-mailadresse. Dette er både med til at forhindre dobbeltoprettelser af brugere og til at sikre databasens konsistens.

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  role user_role NOT NULL  
);
```

Eksempel på UNIQUE constraint på email i users-tabellen. Dette sikrer, at der ikke kan oprettes flere brugere med samme e-mailadresse.

Databasen er designet med fokus på 3. normalform, hvilket betyder, at redundante data er reduceret ved at opdele information i separate, relaterede tabeller. Dette ses blandt andet i tabellerne `roof_style` og `materials`, hvor information kan genbruges på tværs af flere ordrer frem for at blive gemt flere steder i databasen. Ved at organisere data på denne måde bliver databasen mere overskuelig og lettere at vedligeholde.

I tabellen `roof_style` er der dog valgt ikke fuldt ud at følge 3. normalform i forhold til attributten `color`. Dette skyldes, at der kun findes et meget begrænset antal tagfarver i systemet, og det derfor ikke blev vurderet nødvendigt at oprette en separat tabel til håndtering af farver.

Dette valg blev truffet ud fra en vurdering af balancen mellem kompleksitet og funktionalitet, da en yderligere opdeling ville have gjort databasestrukturen mere omfattende uden at skabe væsentlige fordele for systemet. Samtidig har dette været et bevidst designvalg gennem hele udviklingsprocessen, og strukturen er derfor udarbejdet på en måde, hvor den relativt nemt ville kunne ændres til fuld 3. normalform senere, hvis systemets behov skulle ændre sig.

Tabellen `order` fungerer som en central tabel i databasen, da den repræsenterer brugerens forespørgsel i systemet. Tabellen fungerer som et bindeled mellem flere af de øvrige tabeller og samler de nødvendige oplysninger omkring en ordre. Gennem foreign keys kobles "order" blandt andet sammen med `contact_info`, `specifications` og `materials`. Dette gør det muligt at samle relevante data uden at gemme de samme oplysninger flere steder i databasen.

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  contact_info INTEGER REFERENCES contact_info(id),  
  specifications INTEGER REFERENCES specifications(id),  
  workshop INTEGER REFERENCES workshop(id),  
  status order_status NOT NULL,  
  price NUMERIC,  
  uuid UUID  
);
```

Udsnit af `orders`-tabellen, hvor foreign keys bruges til at koble en ordre sammen med kontaktoplysninger, specifikationer og eventuelt redskabsskur.

En af de mere centrale relationer i databasen er mange-til-mange relationen mellem tabellerne `order` og `materials`. Denne relation opstår, fordi én ordre kan indeholde flere forskellige materialer, samtidig med at det samme materiale kan anvendes i flere forskellige ordrer. For at håndtere denne relation er forbindelsestabellen `material_list` blevet oprettet. Tabellen indeholder foreign keys til både `order` og `materials` samt attributten `amount`, som angiver mængden af det pågældende materiale i den specifikke ordre.

```
CREATE TABLE material_list (
  order_id INTEGER REFERENCES orders(id),
  material_id INTEGER REFERENCES materials(id),
  amount INTEGER NOT NULL,
  length_mm INTEGER NOT NULL,
  description TEXT,
  PRIMARY KEY (order_id, material_id, length_mm)
);
```

Forbindelsestabelen `material_list`, som gør det muligt at koble flere materialer til én ordre og genbruge samme materiale på tværs af flere ordrer.

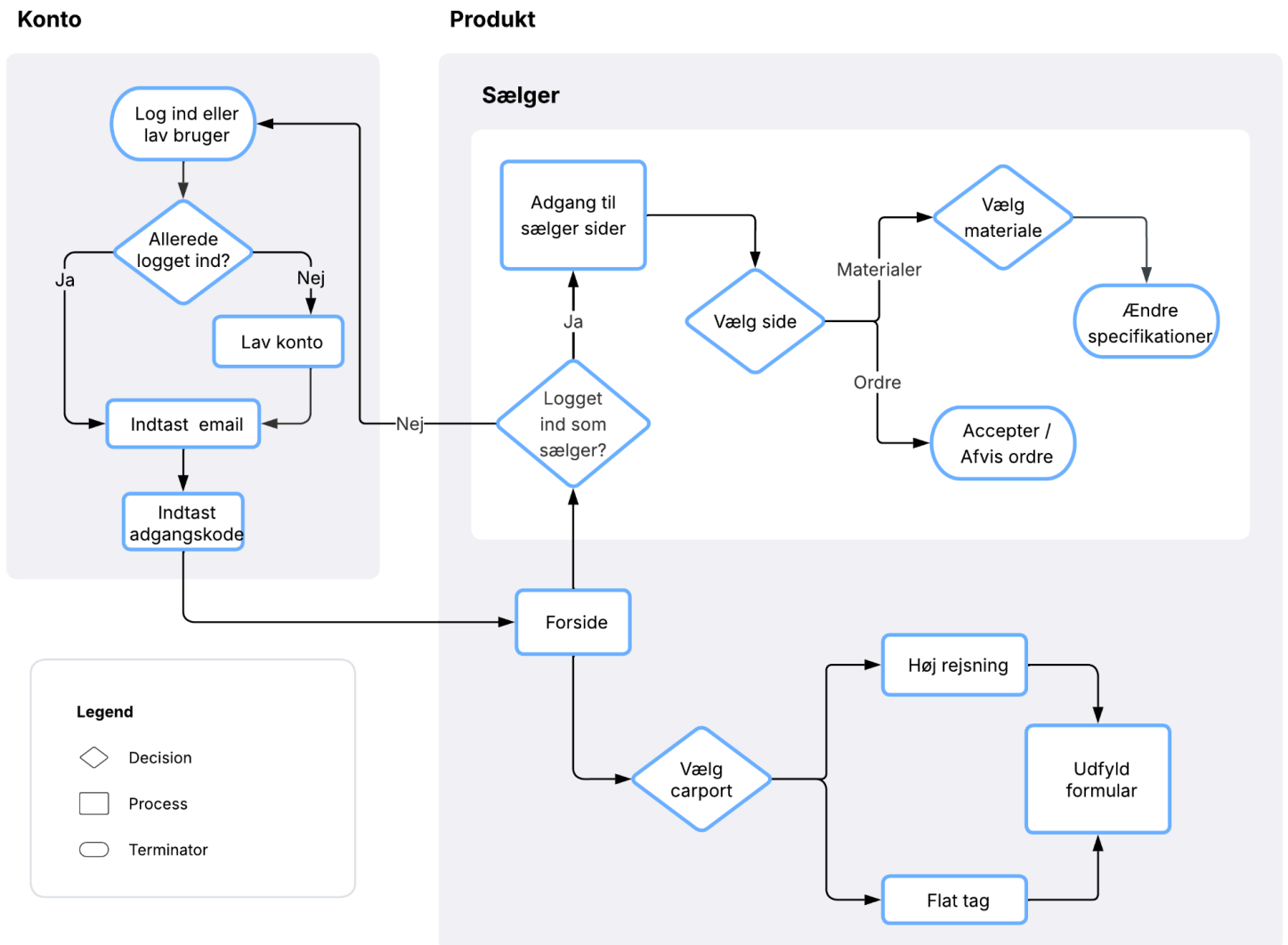
Ved at implementere relationen gennem en separat forbindelsestabel undgås redundans i databasen, da materialerne kun behøver at blive gemt ét sted. Samtidig gør denne løsning databasen mere fleksibel og lettere at vedligeholde, da nye materialer kan tilføjes og genbruges på tværs af flere ordrer uden ændringer i den øvrige databasestruktur.

Tabellen `contact_info` er adskilt fra `user`-tabellen for at skabe en tydelig opdeling mellem brugerens loginoplysninger og personlige kontaktoplysninger. Tabellen `user` indeholder information relateret til autentifikation og brugerroller, mens `contact_info` indeholder oplysninger som navn, adresse og telefonnummer. Denne opdeling bidrager til en mere overskuelig og normaliseret databasestruktur, hvor data er organiseret efter deres ansvarsområde.

Samtidig gør løsningen databasen mere fleksibel og skalerbar, da kontaktoplysninger senere kan udvides uden større ændringer i `user`-tabellen. Derudover bidrager adskillelsen også til bedre vedligeholdelse og sikkerhed, da loginrelaterede oplysninger holdes adskilt fra øvrige brugerdata.

Databasen er overordnet designet med fokus på normalisering, fleksibilitet og fremtidig udvidelse af systemet. Ved at opdele data i separate relaterede tabeller reduceres redundans, samtidig med at databasen bliver lettere at vedligeholde. Denne struktur gør samtidig systemet mere skalerbart, da nye materialer, tagtyper og specifikationer kan tilføjes uden større ændringer i den eksisterende databasestruktur. Databasen er dermed designet på en måde, der både understøtter systemets nuværende funktionalitet og giver mulighed for videreudvikling i fremtiden.

Navigationsdiagram & Mockups



Vi har valgt at dele navigationsdiagrammet op i tre dele. Til venstre ses *Konto-boksen*, som viser login-flowet for systemets brugere. Denne del dækker blandt andet oprettelse af bruger, login og logout. Flowet er relevant for både kunder og sælgere, da begge typer brugere kan have en konto i systemet.

Til højre ses *Produkt-boksen*, som viser den overordnede navigation i selve carport-flowet. Denne del starter på forsiden, hvor brugeren kan vælge carporttype. Herfra kan kunden gå videre til formularen for enten fladt tag eller høj rejsning. I den nuværende version er det dog kun formularen til fladt tag, der er fuldt implementeret. Når kunden udfylder og indsender formularen, sendes data videre til systemet, hvor forespørgslen oprettes og efterfølgende kan behandles af en sælger. Dette hænger især sammen med *User Story 4*, hvor kunden skal kunne vælge carporttype og sende en forespørgsel.

Inde i Produkt-boksen har vi placeret den tredje del, *Sælger-boksen*. Denne viser navigationsflowet for brugere med sælgerrettigheder. Sælgere har adgang til dele af systemet, som almindelige kunder ikke har adgang til. Her kan sælgeren se en oversigt over forespørgsler, filtrere ordrer efter status, se detaljer, godkende eller afvise forespørgsler samt ændre data på materialer og priser. Dette hænger blandt andet sammen med *User Story 3*, *User Story 6*, *User Story 7* og *User Story 8*.

Navigationen er implementeret gennem Javalin-routes og Thymeleaf-skabeloner. I `Main` registreres forsiden på `/`, mens de øvrige routes registreres gennem controller-klasserne `UserController`, `SalesController` og `PaymentController`. For eksempel håndterer `UserController` login, brugeroprettelse og carportformularer, mens `SalesController` håndterer sælgerens ordre- og materialeadministration.

Funktion	Route	Template	Adgang
Forside / valg af carport	/	fog-carport.html	Alle
Login	/login-page	login.html	Alle
Opret bruger	/create-user	create-user.html	Alle
Carport med fladt tag	/carport/flat-roof	flat-roof.html	Alle
Indsend formular	/carport/send-form	-	Alle
Sælgerens ordreoversigt	/admin/orders	orders.html	Sælger
Materialeadministration	/admin/materials	material-admin.html	Sælger
Betalingsside	/payment	payment.html	Kunde med betalingslink

Sælger-siderne er beskyttet gennem `AuthFilter`, som kontrollerer alle routes under `/admin/`. Hvis brugeren ikke er logget ind med rollen `EMPLOYEE`, vises en fejlside med statuskode 403. På den måde sikrer systemet, at almindelige kunder ikke kan tilgå sælgerens ordreoversigt eller materialeadministration.

Systemet benytter desuden en fælles header/navigation på de fleste sider. I denne navigation vises links som “Se alle ordre” og “Materialer” kun, hvis brugeren er logget ind som `EMPLOYEE`. Dermed tilpasser navigationen sig brugerens rolle, så sælgeren får adgang til administrative funktioner, mens kunden kun ser de almindelige dele af systemet.

Med hensyn til mockups har vi ikke haft behov for at udarbejde selvstændige mockups i større omfang. Det skyldes, at vi valgte at tage udgangspunkt i Fogs eksisterende hjemmeside og dens visuelle stil. Formålet var, at vores system skulle føles som en naturlig del af Fogs nuværende hjemmeside. Derfor har vi fokuseret på at genskabe layout, farver, navigation og formularstruktur, så brugeroplevelsen blev så ensartet som muligt.

Valg af arkitektur

Vi har valgt en MVC-inspireret arkitektur, da det er den arkitektur vi er blevet undervist i og har erfaring med. MVC-arkitekturen skaber en klar adskillelse af ansvar mellem de tre lag - Model, View og Controller, hvilket gør koden mere overskuelig og lettere at vedligeholde og udvide.

Vi har ikke implementeret en fuldstændig ren MVC-arkitektur, men en variant der læner sig op ad det vi har lært i undervisningen.

Packages og struktur

app.controllers

Controller-laget i MVC. Indeholder vores Javalin routes og håndterer HTTP-requests. Controllers modtager input fra brugeren, kalder de relevante services og mappere, og sender data videre til view-laget via `ctx.attribute()` og `ctx.render()`. Vi har opdelt controllers efter ansvarsområde. `UserController` håndterer brugerrelaterede routes, `SalesController` håndterer sælgerrelaterede routes og `PaymentController` håndterer betalingsflow.

app.entities

Indeholder alle entitetsklasser der repræsenterer systemets domæneobjekter:

`Order`, `ContactInfo`, `Specifications`, `Workshop`, `Material`, `User` osv.

Disse klasser er bygget med Builder-mønsteret, som sikrer at objekter altid er gyldigt oprettet med alle nødvendige felter.

Vi har brugt builder mønsteret, da det ikke i alle tilfælde vil være relevant at have alle variabler sat i en given instans af de forskellige klasser. Vi har draget fordel af builder mønsterets fleksibilitet, i stedet for at skulle overloade klassernes konstruktører.

app.persistence

Indeholder alle mapper-klasser, der kommunikerer direkte med databasen via JDBC.

Hver mapper har ansvar for én tabel eller et afgrænset domæne — `OrderMapper`, `UserMapper`, `MaterialsMapper` osv. Mapper-klasserne indeholder kun SQL-logik og ingen forretningslogik.

I vores mapper-klasser har vi anvendt `StringBuilder` til dynamisk at bygge SQL UPDATE-statements. Dette gør det muligt at opdatere kun de felter der faktisk er ændret, frem for at overskrive alle felter hver gang. Teknikken bruges i fx

`OrderMapper.changeOrderDetails()` og

`MaterialsMapper.updateMaterialInfo()`, hvor vi itererer over de felter der ikke er null og bygger SQL-strengen op dynamisk. Dette reducerer unødvendige databaseskrivninger og gør metoderne mere fleksible.

app.service

Indeholder serviceklasser der koordinerer forretningslogik på tværs af mappere.

`OrderService` håndterer fx. oprettelse af ordrer ved at kalde flere mappere i rækkefølge. `MaterialService` er en abstrakt klasse der definerer skabelonen for materialberegning, med subklasserne `StandardFlatRoofCalculator` og `StandardRaisedRoofCalculator` som implementerer beregningslogikken for de to tagtyper, og beregner materialerne baseret på hver classes interne logik.

app.exceptions

Indeholder custom exception-klasser som `DatabaseException` der bruges til at håndtere databasefejl på en kontrolleret måde, så de kan præsenteres som brugervenlige fejlbeskeder.

app.config

Indeholder konfigurationsklasser. `SessionConfig` til sessionhåndtering og

`ThymeleafConfig` til opsætning af template engine.

resources/templates

View-laget i MVC. Indeholder alle Thymeleaf HTML-templates der præsenterer data for brugeren.

resources/static

Indeholder statiske filer som CSS og JavaScript.

Design patterns

Singleton - ConnectionPool

`ConnectionPool` er implementeret som et Singleton, hvilket sikrer at der kun eksisterer én databaseforbindelsespulje i hele applikationen. Instansen oprettes første gang `getInstance()` kaldes og genbruges derefter. Dette er centralt for at undgå unødvendige databaseforbindelser og for at have kontrol over ressourceforbruget.

Builder - entitetsklasser

Alle entitetsklasser som `Order`, `Specifications` og `ContactInfo` bruger Builder-mønsteret. Dette valg er truffet fordi disse klasser har mange felter, hvoraf nogle er valgfrie. Builder-mønsteret sikrer at objekter altid er i en gyldig tilstand når de oprettes, og gør koden mere læsbar end konstruktører med mange parametre.

Template Method - MaterialService

`MaterialService` er en abstrakt klasse der definerer skabelonen for materialberegning med `finalizeOrder()` som hovedmetoden. Subklasserne `StandardFlatRoofCalculator` og `StandardRaisedRoofCalculator` implementerer de konkrete beregningsmetoder. Dette mønster er valgt fordi fladt tag og høj rejsning deler den samme overordnede beregningsstruktur, men adskiller sig i de konkrete beregninger.

Observer - mailnotifikationer

Mailnotifikationer er implementeret med Observer-mønsteret. `OrderObserver` er et interface som definerer kontrakten for alle observers. `CustomerEmailObserver` og `SalesEmailObserver` implementerer interfacet og håndterer henholdsvis mails til kunden og sælgeren. Når en ordres status ændres, notificeres alle registrerede observers automatisk. Dette sikrer at maillogikken er løst koblet fra forretningslogikken og at nye notifikationstyper kan tilføjes uden at ændre i den eksisterende kode.

Factory - materialberegning

`FormService` fungerer som en Factory der ud fra tagtypen returnerer den korrekte `MaterialService` implementering, enten `StandardFlatRoofCalculator` eller `StandardRaisedRoofCalculator`.

Hjælpeklasser

`AuthFilter` håndterer adgangskontrol via Javalin before-filtre og sikrer at beskyttede routes kun er tilgængelige for brugere med de rette rettigheder.

Enumerations

Vi har anvendt enums til at repræsentere begrænsede sæt af værdier i systemet. `OrderStatus` repræsenterer en ordres mulige tilstande (`PENDING`, `OFFER_SENT`, `PAID`, `REJECTED`, `CANCELLED`), `UserRole` repræsenterer brugerroller (`CUSTOMER`, `EMPLOYEE`) og `RoofType` repræsenterer tagtyper (`FLAT`, `RAISED`). Disse enums er defineret både i Java og som PostgreSQL enum-typer i databasen, hvilket sikrer dataintegritet på databaseniveau og typesikkerhed i koden. For at forenkle konverteringen mellem Java og PostgreSQL har vi tilføjet en `getDatabaseEnum()` metode direkte på enum-klasserne, som returnerer et `PGObject` med den korrekte PostgreSQL type. Dette betyder at mapperne ikke behøver at kende til konverteringslogikken.

Records

Vi har anvendt Java records til simple dataklasser der udelukkende skal holde på data uden forretningslogik. `OrderDetails` og `MaterialListEntry` er eksempler på records i projektet. Records genererer automatisk konstruktør, getters, `equals()`, `hashCode()` og `toString()`, hvilket reducerer mængden af boilerplate-kode sammenlignet med normale klasser. Vi har valgt records frem for normale klasser når objektet er immutable og kun tjener som databeholder.

Særlige forhold

Session og request variabler

Session-variablen "user" indeholder et `User`-objekt med brugerens id, e-mail, hashed password og rolle. Den sættes ved login og bruges i `AuthFilter` til adgangskontrol. På hvert request injiceres brugeren fra sessionen som request-attribut så Thymeleaf-templates kan tilgå den.

Øvrige request-attributter sat af controllers inden rendering: "errorMessage" og "successMessage" til feedback, "orders" og "materials" til admin-panelet, "widths", "lengths", "workshopWidths", "workshopLengths" til ordreformularen, samt "tiles" og "statusFilter" til filtreret visning.

Exception-håndtering og logging

Projektet anvender tre custom exception-klasser: `DatabaseException`, `InvalidCredentialsException` og `UserNotFoundException`. Disse fanges i controllers og brugeren præsenteres for generiske fejlbeskeder for at undgå at lække systemdetaljer. Login-fejl returnerer bevidst den samme besked uanset om det er e-mail eller password der er forkert, for at forhindre bruger-enumeration.

Logging sker med `System.err.println()` i controllers og services med klassens og metodens navn som præfiks, fx `[UserController.createUser]`. `ConnectionPool` bruger `java.util.logging.Logger` til at logge oprettelse og nedlukning af connection-poolen.

Brugerinput-validering

Backend-validering er centraliseret i `OrderFormService.validateOrderForm()`. Tekstfelter tjekkes med `requireNotBlank()`, numeriske felter med `requireInt()`, og roofType valideres mod enum-værdierne `FLAT` og `RAISED`. Login-inputs valideres separat i `UserController.validateLoginInputs()`. Frontend-validering via HTML5-attributter, som `required` og `type="email"`, understøtter backend-valideringen, men den egentlige sikkerhedsmæssige validering foregår altid server-side.

Sikkerhed og login

Adgangskoder hashes med BCrypt ved brugeroprettelse og verificeres ved login. Det rene password gemmes aldrig i databasen.

`AuthFilter` beskytter alle routes under `/admin/*` ved at tjekke sessionen på hvert request. Brugere uden `EMPLOYEE`-rollen modtager HTTP 403.

Brugerroller

Projektet har en `Role`-enum med tre værdier: `CUSTOMER` der kan oprette og se egne ordrer, `EMPLOYEE` der kan se og behandle alle ordrer i admin-panelet, og `ADMIN` der er defineret i koden men endnu ikke taget i brug. Rollen gemmes i databasen som et PostgreSQL enum-type og konverteres i JDBC via `Role.getDatabaseEnum()` til det tilsvarende `PGObject`.

Tegning

Carport-tegningen genereres programmatisk som SVG og vises direkte i browseren. `Svg` bygger SVG-markup med metoder som `addRectangle()`, `addLine()` og `addText()`. `SvgCalculationService` beregner geometri ud fra carportens mål med konstanter som maks afstand mellem stolper (3000 mm) og overhæng (1000 mm). `SvgDrawingService` orkestrerer tegningen og genererer en side- og topvisning med målstreger og tekstannotationer. Koordinater skales fra millimeter til SVG-pixels.

Styklisteberegner

Styklistens logik er implementeret med Template Method Pattern i den abstrakte klasse `MaterialService`. `forRoofType()` fungerer som factory og returnerer den rigtige implementation — pt. `StandardFlatRoofCalculator` for flade tage. Beregningerne baserer sig på carportens dimensioner og indsættes i databasen via `MaterialsMapper.insertMaterialList()`.

Email-notifikationer og PDF

Projektet bruger Observer Pattern til at sende emails ved statusændringer på ordrer. `CustomerEmailObserver` sender emails til kunden ved alle statusskift, og `SalesEmailObserver` notificerer salgsteamet ved nye ordrer. `OrderService` vedligeholder en liste af observers og kalder `notifyObservers()` ved hvert statusskift.

Al email-afsendelse sker asynkront i separate tråde for ikke at blokere HTTP-svaret til brugeren.

Ved `OFFER_SENT` genereres et UUID-token der inkluderes i betalingslinket så betalingen kan verificeres uden login.

Ved `PAID` genereres en PDF med to sider: SVG-tegningen konverteres via Apache Batik, og styklisten renderes som HTML-tabel via Flying Saucer. De to PDFs merges til én fil med PDFBox og vedhæftes bekræftelsesmailen.

Deployment

Applikationen er hostet på en DigitalOcean Droplet med Docker Compose. Caddy fungerer som reverse proxy og håndterer HTTPS automatisk via Let's Encrypt. Applikationen er tilgængelig på carport.mikkelmattern.dk

For at give et samlet overblik over de vigtigste klasser og deres relationer har vi placeret et klassediagram i bilaget. Klassediagrammet viser blandt andet sammenhængen mellem centrale entitetsklasser som `Order`, `ContactInfo`, `Specifications`, `Workshop`, `Material` og `User`, samt hvordan disse klasser understøtter systemets ordreflow. Diagrammet fungerer derfor som et supplement til domænemodellen og ER-diagrammet, men viser systemet fra et mere implementeringsnært perspektiv.

Udvalgte kodeeksempler

```
public class CustomerEmailObserver implements OrderObserver {
    EmailSender emailSender = new EmailSender();

    @Override
    public void update(Order order, OrderStatus status, ConnectionPool connectionPool) {
        String email = order.getContactInfo().getEmail();
        String subject = "";
        String body = "";
        String customerName = order.getContactInfo().getFirstName() + " " + order.getContactInfo().getLastName();
        String baseUrl = System.getenv("BASE_URL") != null ?
            System.getenv("BASE_URL") : "http://localhost:7070";

        int orderId = order.getId();
        UUID uuid = UUID.randomUUID();
        String token = uuid.toString();

        try {
            OrderMapper.changeOrderDetails(orderId, new OrderDetails(uuid), connectionPool);
            switch (status) {
                case PENDING -> {
                    String[] strings = handlePendingStatus(customerName, orderId);
                    subject = strings[0];
                    body = strings[1];
                }
                case OFFER_SENT -> {...}
                case CANCELLED -> {...}
                case PAID -> {...}
                case REJECTED -> {...}
            }
            String finalSubject = subject;
            String finalBody = body;
            new Thread(() -> {
                try {
                    emailSender.sendPlainTextEmail(email, finalSubject, finalBody);
                } catch (MessagingException e) {
                    System.err.println("[CustomerEmailObserver.update] Could not send email to " + email + ": " + e.getMessage());
                }
            }).start();
        } catch (DatabaseException e) {
            System.err.println("[CustomerEmailObserver.update] Something went wrong when trying to access the database " +
                e.getMessage());
        }
    }
}
```

OrderObserver – Observer Pattern

update-metoden er implementeringen af Observer Pattern i systemet.

CustomerEmailObserver og SalesEmailObserver implementerer OrderObserver-interfacet og reagerer på statusskift i en ordre.

Når en ordres status ændres, kaldes update med den nye status, dette betyder at observeren ikke behøver kende til hvem der kaldte den eller hvorfor. Den reagerer blot på hændelsen. Dette løskobler maillogikken fra resten af systemet.

Metoden bruger en switch på OrderStatus til at delegere til specifikke håndteringsmetoder per status. PAID er et særtilfælde der håndteres separat, da den indeholder PDF-generering og mailvedhæftning. Til den har vi oprettet en anden metode sendEmailWithPdf i GmailEmailSender. De øvrige statusser sender en simpel tekstmail via en fælles sti nederst i metoden.

Factory-inspireret struktur ses i de private hjælpemetoder (`handlePendingStatus`, `handleOfferSentStatus` osv.) der hver returnerer `subject` og `body` for deres respektive status. Al beskedlogik er centraliseret og let at vedligeholde.

```
public abstract class MaterialService {

    MaterialsMapper materialsMapper = new MaterialsMapper();

    public static MaterialService forRoofType(RoofType roofType, ConnectionPool connectionPool)
        throws UnsupportedOperationException, DatabaseException {

        return switch (roofType) {
            case FLAT -> new StandardFlatRoofCalculator(connectionPool);
            case RAISED ->
                throw new UnsupportedOperationException("Beregning for rejst tag er ikke implementeret endnu");
        };
    }

    public void finalizeOrder(int orderId, Specifications specifications, ConnectionPool connectionPool) throws DatabaseException {
        List<MaterialType> materialTypes = getMaterialTypes();
        insertAllMaterials(materialTypes, orderId, specifications, connectionPool);
    }

    protected void insertAllMaterials(List<MaterialType> materialTypes, int orderId,
        Specifications specifications, ConnectionPool connectionPool) throws DatabaseException {

        for (MaterialType materialType : materialTypes) {
            insertMaterialLine(materialType, orderId, specifications, connectionPool);
        }
    }
}
```

MaterialService – Factory og Template Method Pattern

`MaterialService` er en abstrakt klasse der kombinerer to designmønstre: Factory Method og Template Method.

`forRoofType` er en Factory Method der returnerer den korrekte implementation baseret på tagtype. Når systemet skal beregne materialer til en ordre, kaldes metoden med en `RoofType` og returnerer den rigtige subklasse. Kalderen behøver ikke kende til `StandardFlatRoofCalculator` eller fremtidige implementationer, kun til `MaterialService`. Tilføjelse af en ny tagtype kræver kun en ny subklasse og en ny case i switch-udtrykket.

`finalizeOrder` implementerer Template Method Pattern og definerer skelettet for hvordan materialer beregnes og indsættes i databasen. Metoden kalder `getMaterialTypes` som er abstrakt og implementeres af hver subklasse. Det er her subklassen definerer hvilke materialetyper der er relevante for netop den tagtype.

Selve beregnings- og indsætningslogikken sker i `insertMaterialLine`, der looper over de returnerede typer og bruger et switch-udtryk til at kalde den korrekte beregningsmetode for hver type, fx `calculatePosts` for stolper og

`calculateRafters` for spær. Resultatet indsættes herefter i databasen via `MaterialsMapper`.

Flowet kan illustreres således:

1. `finalizeOrder` kalder `insertAllMaterials` med listen fra `getMaterialTypes`
2. `insertAllMaterials` looper over hver materialetype som er defineret i de konkrete klasser
3. `insertMaterialLine` beregner antal for én type
4. `calculate*` udfører den konkrete beregning

Arkitekturen sikrer at beregningsflowet er ens for alle tagtyper, mens subclasses kun varierer det der må variere: hvilke materialer der beregnes, og hvordan antallet udregnes.

```
@Override
protected int[] calculatePosts(Specifications specifications) {
    int roomBetween = specifications.getLengthCm() - (2 * 100);
    int sections = (int) Math.ceil((double) roomBetween / 300);
    return new int[] {
        (sections + 1) * 2,
        materials.get(MaterialType.POST).getMaxLength() / 10
    };
}
```

Udsnit fra `StandardFlatRoofCalculator`, hvor antallet af stolper beregnes ud fra carportens længde.

Metoden `calculatePosts()` viser et eksempel på, hvordan systemet omsætter kundens carportmål til konkrete materialer i styklisten. Først fratrækkes overhænget i begge ender af carporten, så der kun regnes på det område, hvor stolperne skal placeres. Derefter beregnes antallet af sektioner med `Math.ceil()`, så afstanden mellem stolperne ikke bliver for stor.

Til sidst returneres både antallet af stolper og længden på materialet. Da stolperne placeres på begge sider af carporten, ganges antallet med 2. Denne beregning understøtter især *User Story 4*, hvor kundens indtastede mål skal kunne bruges til at danne en forespørgsel og beregne materialer.

Status på implementering

Implementerede funktioner

På afleveringstidspunktet er de centrale dele af systemets kunde- og sælgerflow implementeret. Systemet understøtter både oprettelse af forespørgsler, behandling af ordrer, e-mailnotifikationer, materialeadministration og generering af stykliste og tegning efter betaling. De implementerede funktioner er:

- Kunder kan oprette bruger og logge ind.
- Kunder kan vælge carporttype og sende en ordre/forespørgsel.
- Kunder modtager en e-mail, når de har oprettet en ordre.
- Sælger modtager en e-mail, når der kommer en ny ordre.
- Sælger kan logge ind og se alle ordrer med relevante informationer.
- Sælger kan filtrere ordrer efter status.
- Sælger kan vælge en ordre, der endnu ikke er behandlet, og acceptere den.
- Kunden modtager en e-mail, når ordren er blevet accepteret.
- Kunden modtager et betalingslink på mail.
- Kunden kan afvise eller betale ordren, efter tilbuddet er blevet accepteret.
- Kunden modtager en mail med stykliste og tegning, når ordren er betalt.
- Sælger kan se materialer og ændre deres pris, navn og beskrivelse.
- Der er ikke implementeret en rigtig betalingsintegration. Betalingen er i stedet simuleret med en knap, der markerer ordren som betalt.

De implementerede funktioner dækker især de centrale user stories for kundeforespørgsel, sælgerbehandling, tilbud, betaling og stykliste. Det gælder blandt andet *User Story 4*, hvor kunden sender en forespørgsel, *User Story 8* og *User Story 9*, hvor sælgeren sender et tilbud og kunden accepterer eller afviser det, samt *User Story 10*, hvor kunden efter betaling modtager kvittering, stykliste og tegning.

Kendte mangler

Selvom systemet har et funktionelt hovedflow, er der stadig en række funktioner, som ikke er færdigimplementeret. De kendte mangler er:

- Kunden kan ikke vælge carport med høj rejsning, da der ikke er lavet beregninger eller en færdig klasse til denne funktion.
- Kunden kan ikke vælge carport med redskabsskur, da der ikke er lavet funktionalitet til at tegne dette i SVG eller udregne materialerne.
- Der er ingen funktion til "glemt adgangskode".
- Kunden kan ikke ændre sin adgangskode.
- Kunden kan ikke se deres tidligere ordrer.
- UUID-tokenet til betalingslinket virker på ubestemt tid.

De kendte mangler betyder, at systemet endnu ikke understøtter alle de funktioner, der var tænkt i den fulde løsning. Særligt høj rejsning og redskabsskur er relevante udvidelser, fordi de ville gøre carportkonfiguratoren mere komplet. Disse mangler er dog afgrænsede, og systemets primære flow fra forespørgsel til betalt ordre og styklister fungerer.

Kvalitetssikring (test)

Autoriserede tests

Vi har to typer tests i vores projekt: integration tests og unit tests.

Integration tests verificerer samspillet mellem vores persistens lag (java-kode) og databasen. Testene kører mod en dedikeret testdatabase, der nulstilles før hver test. Dette sikrer isolation og gør testene uafhængige af hinanden.

Test arkitektur

Vores integration tests arver fra `DatabaseTest`. Denne baseklasse har to ansvarsområder. For det første nulstilles databaseskemaet før hver test, så ingen test efterlader data. For det andet leverer klassen en delt `ConnectionPool`, som mapperne bruger til at hente forbindelser.

`ConnectionPoolTest` validerer forbindelsespuljens adfærd.

`successGetInstance()`

Denne metode tester at puljen implementerer Singleton-mønsteret korrekt. Hvis `getInstance()` returnerede forskellige objekter, ville vi risikere at oprette for mange forbindelser til databasen. Testen kalder `getInstance()` to gange og bruger `assertSame()` til at verificere at begge referencer peger på samme objekt.

`failGetInstance()`

Denne metode tester at puljen håndterer ugyldige input korrekt. Hvis man kalder `getInstance()` med "null"-parametre, bør systemet kaste en exception frem for at forsøge at oprette en pulje med ugyldige credentials. Testen kalder `resetInstance()` for at sikre en frisk tilstand. Derefter kaldes `getInstance()` med "null"-parametre. Testen bruger `assertThrows()` til at bekræfte at en exception bliver kastet.

`getConnection()`

Denne metode tester at puljen kan udlevere en brugbar forbindelse. En forbindelse der er null eller lukket ville få hele applikationen til at fejle. Testen kalder `getConnection()` som returnerer et `Connection`-objekt. Derefter bruges `assertNotNull()` til at bekræfte at objektet ikke er null. `assertFalse()` kaldes med `isClosed()` for at bekræfte at forbindelsen er åben. Til sidst bruges `assertTrue()` med `isValid(2)` for at bekræfte at forbindelsen kan kommunikere med databasen inden for 2 sekunder.

`resetInstance()`

Denne metode tester at man kan nulstille Singleton-instansen. Dette er vigtigt i testmiljøer, hvor man ønsker en frisk pulje mellem testkørsler. Testen gemmer først den eksisterende instans i en variabel. Derefter kaldes `resetInstance()` som nulstiller Singleton'en. Herefter oprettes en ny instans med `getInstance()`. Til sidst bruges `assertNotSame()` til at bekræfte at den gamle og den nye instans er forskellige objekter.

`close()`

Denne metode tester at en lukket pool ikke længere kan udlevere forbindelser. Hvis man fortsat kunne hente forbindelser efter `close()`, ville det føre til memory leaks og ustabile forbindelser. Testen kalder `close()` på puljen. Derefter kaldes `getConnection()` og `assertThrows()` bekræfter at en `SQLException` bliver kastet. Til sidst nulstilles puljen igen med `resetInstance()` og `getInstance()` så efterfølgende tests ikke fejler.

`WorkshopMapperTest`

Tester persistensoperationer for workshop-entiteten.

`insertWorkshop()`

Denne metode tester at en workshop kan gemmes i databasen og returnerer et genereret ID. Hvis ID'et var 0 eller negativt, ville det indikere at insert-operationen fejlede. Testen opretter først et `Workshop`-objekt med `Builder`-mønsteret. Derefter kaldes `insertWorkshop()` med objektet og `connectionPool`. Metoden returnerer et `int` ID. Testen bruger `assertTrue()` til at verificere at ID er større end 0. For at bekræfte at data er gemt korrekt kaldes `findWorkshopById()` med ID'et. Testen bruger `assertEquals()` til at sammenligne `getWidthCm()` og `getLengthCm()` fra det oprindelige objekt med det gemte objekt.

`findWorkshopById()`

Denne metode tester to scenarier. Først testes happy path: et eksisterende ID skal returnere en gyldig workshop. Testen opretter og gemmer en workshop for at få et rigtigt ID. Derefter kaldes `findWorkshopById()` med ID'et. Testen bruger `assertEquals()` til at verificere at ID, width og length matcher. Derefter testes edge case: et ikke-eksisterende ID skal returnere null. Testen kalder `findWorkshopById()` med -1 som ID. Testen bruger `assertNull()` til at bekræfte at metoden returnerer null.

Unit Test

En unit test er en automatiseret test, der isoleret verificerer, at en enkelt metode eller funktion opfører sig korrekt uden afhængighed af eksterne systemer som database eller netværk.

`StandardFlatRoofCalculatorTest` er vores testklasse, der udelukkende fokuserer på at verificere beregningslogikken i carport-beregneren. Testklassen indeholder flere metoder, hvoraf `calculatePostsShouldAlwaysFollowTheRules()` tester stolpeberegningen. Metoden beregner antallet af stolper ud fra carportens længde. Stolper skal stå 1 meter fra hver ende og derefter med maksimalt 300 cm mellemrum – og altid i lige antal, da de sidder på begge sider. Testen opretter en instans af `StandardFlatRoofCalculator`. Derefter kører en løkke for alle længder fra 200 cm til 2000 cm. For hver længde bygges et `Specifications`-objekt, og `calculatePosts()` kaldes.

Testen verificerer tre forhold: Lige antal: `totalPosts[0] % 2 == 0`, hvis der kommer en rest, er antallet ulige og testen fejler. Maks afstand 300 cm: Testen beregner den faktiske afstand mellem stolperne. Hvis afstanden overstiger 300 cm, fejler testen. Positiv afstand: Hvis der kun er én stolpe på en side, bliver afstanden negativ eller nul. Testen tjekker at afstanden er større end 0. Ved at køre alle længder i én løkke testes samtidig alle de længder, hvor antallet af stolper skifter (omkring 300 cm, 600 cm, 900 cm osv.). Hvis blot én længde bryder en regel, fejler testen. Reglerne i testen findes, fordi stolpeberegningen skal overholde konstruktionskravene til en carport.

Hvis testen ikke tjekkede disse regler, ville vi ikke vide, om beregningen var korrekt. Ved at verificere både lige antal, maksimal afstand på 300 cm og positiv afstand sikrer vi, at metoden følger de fysiske krav – ellers kunne carporten blive ustabil eller umulig at bygge.

Tests giver os tillid til at vores databasekommunikation virker korrekt. Unit tests giver os tillid til at vores beregningslogik er korrekt.

Samlet set udgør testsene et automatiseret regressionstest-suite, der ved hver kodeændring hurtigt identificerer brud på eksisterende funktionalitet.

User Acceptance tests

Til hver user story har vi lavet en user acceptance test, hvor vi har vurderet, om funktionaliteten lever op til de acceptkriterier, der blev formuleret i starten af projektet. Testene er ikke automatiserede, men er gennemført manuelt ved at afprøve de relevante flows i systemet.

User Story	Resultat	Kommentar
1	Bekræftet	Bruger kan oprettes, og fejlbesked vises ved ugyldige eller manglende oplysninger.
2	Bekræftet	Sælger kan logge ind og få adgang til sælgerfunktioner. Ugyldigt login giver fejlbesked.
3	Bekræftet	Sælger kan se materialer og ændre pris, navn og beskrivelse.
4	Delvist bekræftet	Kunden kan vælge carport med fladt tag og sende formularen. Høj rejsning og redskabsskur er ikke fuldt implementeret.
5	Bekræftet	Sælger modtager mail, når en kunde sender en forespørgsel.
6	Bekræftet	Sælger kan se en oversigt over forespørgsler/ordrer.
7	Bekræftet	Sælger kan se detaljer og pris for en forespørgsel.
8	Bekræftet	Sælger kan godkende eller afvise en forespørgsel, og status ændres i systemet.
9	Bekræftet	Kunden modtager tilbudsmail og kan acceptere eller afvise tilbuddet via betalingslinket.
10	Bekræftet	Kunden modtager kvitteringsmail med stykliste og tegning efter simuleret betaling.

Proces

Arbejdsprocessen Faktuelt

I løbet af den første fase af projektet brugte vi størstedelen af tiden på det forberedende arbejde. Dette inkluderede blandt andet udvikling af de diagrammer, som det var muligt at lave på dette tidlige stadie af processen. Derudover brugte vi tid på at udarbejde og formulere vores user stories samt deres tilhørende acceptkriterier. I denne fase arbejdede vi især med at skabe et fælles overblik over systemets funktioner, så vi senere kunne omsætte dem til konkrete tasks.

I anden fase fortsatte vi med det forberedende arbejde. Her arbejdede vi blandt andet med at undersøge, hvilke materialer der skulle bruges til en carport, hvilke mål materialerne havde, og hvordan de første iterationer af vores beregninger skulle se ud. Beregningerne skulle bruges til at finde mængden af de forskellige materialer, der skal bruges til at bygge en carport. Halvvejs gennem denne fase begyndte vi også på opsætningen af vores database, da vi på dette tidspunkt havde fået en bedre idé om, hvilke data systemet skulle indeholde. Vi oprettede desuden tasks i vores KanBan-board med udgangspunkt i de acceptkriterier, vi havde udarbejdet i den første fase.

I tredje fase begyndte vi for alvor på selve implementeringen af projektet. Her satte vi projektstrukturen op, fordelte tasks mellem os og arbejdede på at løse så mange funktionelle dele af systemet som muligt. Vi arbejdede blandt andet med user stories relateret til indtastning af carportmål, beregning af materialer og visning af tegninger. Derudover lavede vi løbende tests for at sikre, at de forskellige dele af systemet fungerede efter hensigten. Slutningen af fasen blev også brugt på at skrive dele af rapporten.

I starten af fjerde fase havde vi et færdigt og funktionelt produkt, men der var stadig en række mindre fejl, som vi løste en efter en. I denne fase arbejdede vi derfor især med fejlrettelser, finpudsning af funktionalitet og forbedring af diagrammer. Derudover brugte vi en stor del af fasen på rapportskrivning, så vi ved slutningen af fasen stod tilbage med et fyldestgørende og funktionelt produkt, som vi var tilfredse med.

I anden fase udnævnte vi Mikkel som KanBan Master (KM). Han stod hovedsageligt for at fordele arbejdsopgaverne mellem gruppens medlemmer og skabe overblik over, hvilke tasks der skulle løses. I praksis blev arbejdsfordelingen dog løbende justeret, da der var mange forskellige opgaver at tage fat på. Derfor endte gruppemedlemmerne i højere grad med selv at vælge, hvilke dele af projektet de ville arbejde med. Dette blev løbende kommunikeret til resten af gruppen, så alle på teamet var opdaterede og vidste, hvad de andre arbejdede på.

Inden vores vejledningsmøder aftalte vi at holde vores eget interne møde. Her planlagde vi, hvem der skulle præsentere hvad, og hvilke emner vi ville tage op med vejlederen. Et eksempel på dette var et vejledningsmøde, hvor vi på forhånd havde planlagt at spørge ind til vores databaseopbygning og vores beregninger af materialer. Under mødet fik vi feedback, som vi efterfølgende brugte til at justere vores videre arbejde og gøre vores tasks mere konkrete.

Den løbende kommunikation i teamet foregik primært gennem daglige møder, hvor vi koordinerede arbejdet i gruppen. På møderne gennemgik vi, hvad de enkelte gruppemedlemmer havde arbejdet på, hvilke opgaver der stadig manglede, og hvad der skulle prioriteres frem mod næste møde. Vi aftalte desuden, hvilke dele af projektet der skulle være færdige til næste dag. På den måde forsøgte vi at sikre, at alle havde et klart overblik over projektets fremgang og vidste, hvad de skulle arbejde videre med.

GitHub blev desuden brugt til versionsstyring, pull requests og til at skabe overblik over tasks gennem vores KanBan-board. Boardet gjorde det lettere at se, hvilke opgaver der var planlagt, i gang eller færdige, og fungerede derfor som et fælles overblik over projektets fremdrift. Et skærmbillede af vores KanBan-board findes i bilaget.

Arbejdsprocessen Reflekteret

Udnævnelsen af en KanBan Mester fungerede generelt godt for vores team. Rollen var med til at skabe struktur for de gruppemedlemmer, som havde brug for et tydeligere overblik over arbejdsopgaverne. Samtidig holdt vi rollen som KanBan Mester relativt dæmpet, så der stadig var plads til, at alle kunne byde ind med, hvad de havde lyst til at arbejde på under vores daglige møder.

Arbejdsfordelingen blev i praksis ikke kun opdelt ud fra konkrete tasks, men i højere grad ud fra funktioner eller emner. Dette fungerede godt, især når nogle gruppemedlemmer havde særlige kompetencer inden for bestemte områder. Det gjaldt blandt andet vores KanBan Mester, som i store træk stod for databasen, da han havde en god forståelse for databaser. Fabian stod for SVG-tegningen, da han havde erfaring med at arbejde med SVG'er, og Nali stod for store dele af testene, da han havde en god forståelse for testarbejde.

Større dele af det forberedende arbejde, såsom diagrammer, arbejdede vi sammen om. Diagrammerne havde stor betydning for, hvordan projektets mål og struktur kom til at se ud. Derfor var det vigtigt for os, at alle på teamet så vidt muligt havde den samme forståelse af, hvad det færdige projekt skulle indeholde, og hvilken retning vi arbejdede hen imod.

I starten havde vi lidt udfordringer med at nedbryde nogle af vores user stories til konkrete tasks, fordi flere af dem dækkede over store dele af systemet. Det gjaldt især *User Story 4*, som både omfattede valg af carporttype, formular, validering, databasegemning, mailafsendelse og PDF-tegning. Vi løste dette ved at tage udgangspunkt i acceptkriterierne og bruge dem som grundlag for mindre tasks i vores KanBan-board. På den måde blev det lettere at fordele arbejdet og følge med i, hvilke dele af en user story der var færdige.

Vi fandt relativt tidligt en god og hensigtsmæssig arbejdsrytme. Dette skyldtes dels vores store fokus på det forberedende arbejde i starten af projektet, men også at vi i teamet havde arbejdet sammen før på tidligere projekter. Det betød, at vi kendte hinandens styrker og svagheder, og derfor lettere kunne kompensere for dem i løbet af processen, hvis det blev nødvendigt.

Vores estimeringer af arbejdet ramte for det meste godt. Dette skyldtes dels, at vores forarbejde gjorde det lettere at vurdere, hvor lang tid hver opgave ville tage, dels at vi i teamet havde en god forståelse for koden, og dels at vi generelt sørgede for at afsætte rigeligt med tid til opgaverne.

De daglige møder gjorde det også lettere at følge op på vores estimeringer. Her kunne gruppe-medlemmerne fortælle, om de forventede at blive færdige til tiden, om en opgave tog længere tid end forventet, eller om de allerede var færdige og klar til at arbejde videre med noget nyt. Denne arbejdsrytme gjorde det lettere at justere arbejdet løbende. Hvis en opgave endte med at tage længere tid end forventet, kunne vi ofte nå at tilpasse planen eller hjælpe hinanden, fordi vi havde afsat ekstra tid til de fleste opgaver.

Med hensyn til den mere tekniske del af processen brugte vi også meget tid i starten af projektet på at sikre, at alt det tekniske fungerede. Det betød, at vi for det meste undgik større tekniske problemer i forbindelse med at dele og merge kode med hinanden via GitHub. Når der opstod mindre problemer, kunne vi som regel løse dem hurtigt, fordi vi havde en fælles forståelse af vores GitHub-workflow.

Med hensyn til vejledningsmøderne fik vi generelt svar på de emner, vi spurgte ind til, samt brugbar feedback. Vi forsøgte også at tage noter under hvert møde, så vi havde vejlederens svar og forslag på skrift. Ved vores sidste vejledningsmøde blev vores vejleder desværre nødt til at aflyse mødet. På dette tidspunkt i processen var vi heldigvis nået så langt, at målstregen var tydeligt i sigte. På trods af dette valgte vi at sætte et møde op med en anden vejleder, som gav os værdifuld feedback. Denne feedback førte til, at vi lavede yderligere fejlrettelser og finjusteringer af projektet.

Bilag

User stories

1. Som sælger eller kunde ønsker jeg at kunne oprette mig i systemet, så jeg kan få adgang til systemets funktioner.

- Givet at jeg er på registreringssiden, når jeg udfylder alle påkrævede felter korrekt, så oprettes jeg som bruger i databasen.
- Givet at jeg udfylder formularen, når nogle påkrævede felter mangler, så vises en fejlbesked.
- Givet at jeg indtaster oplysninger på en eksisterende bruger, når jeg forsøger at registrere mig, så vises en fejlbesked om at brugeren allerede findes.
- Givet at jeg opretter mig korrekt, når registreringen gennemføres, så modtager jeg en bekræftelsesmail.

2. Som sælger ønsker jeg at kunne logge ind, så jeg får adgang til sælger-funktioner.

- Givet at jeg er på login-siden, når jeg indtaster korrekt brugernavn og adgangskode, så bliver jeg logget ind og sendt til overview-siden.
- Givet at jeg indtaster forkert login, når jeg forsøger at logge ind, så vises en fejlbesked.
- Givet at jeg er logget ind som sælger, når jeg navigerer i systemet, så forbliver jeg logget ind.
- Givet at jeg er logget ind som sælger, når jeg forsøger at tilgå sider uden rettigheder, så bliver jeg afvist.
- Givet at jeg er logget ind, når jeg klikker på log-out, så bliver jeg logget ud og mister adgang til beskyttede sider.

3. Som sælger ønsker jeg at kunne ændre priser på materialer, så priserne matcher firmaets ønsker.

- Givet at jeg er logget ind som sælger, når jeg åbner prisoversigten, så kan jeg se alle materialer og deres priser.
- Givet at jeg vælger et materiale, når jeg åbner edit-siden, så kan jeg ændre prisen.
- Givet at jeg indtaster en ny pris, når prisen er ugyldig (fx negativ), så vises en fejlbesked.
- Givet at jeg indtaster en gyldig pris, når jeg gemmer ændringen, så opdateres prisen i databasen.
- Givet at prisen er opdateret, når systemet anvender priser, så bruges den nye pris i beregninger.

4. Som kunde ønsker jeg at vælge carport type og udfylde en formular med mine ønskede specifikationer, så jeg kan sende en forespørgsel på en carport.

- Givet at jeg er på startside, når siden vises, så kan jeg vælge mellem fladt tag og høj rejsning.
- Givet at jeg vælger en carport type, når jeg fortsætter, så sendes jeg videre til den relevante side.
- Givet at jeg er på formular side, når jeg udfylder gyldige mål og specifikationer, så kan jeg sende forespørgslen.
- Givet at jeg vælger skur, når formularen vises, så får jeg mulighed for at angive skurets mål.
- Givet at jeg indtaster manglende eller ugyldige oplysninger, når jeg forsøger at sende formularen, så vises en fejlbesked.
- Givet at formularen er korrekt udfyldt, når jeg sender forespørgslen, så gemmes forespørgslen i databasen og en mail bliver sendt til en sælger og mig selv som kunde.
- Givet at jeg sender en forespørgsel, når den gemmes, så modtager jeg en kvitteringsmail, som indeholder en opsummering og PDF-tegning.

5. Som sælger ønsker jeg at modtage en mail, så jeg kan behandle forespørgsler.

- Givet at en kunde sender en forespørgsel, når den gemmes, så modtager sælgeren en mail.
- Givet at sælger klikker på linket, når han er logget ind, så åbnes forespørgslen.
- Givet at sælger ikke er logget ind, når linket åbnes, så bliver han bedt om login først.
- Givet at linket er ugyldigt, når det åbnes, så vises en fejlbesked.

6. Som sælger ønsker jeg at se alle forespørgsler, så jeg kan give tilbud.

- Givet at jeg er sælger, når jeg åbner oversigten, så vises alle forespørgsler.
- Givet at jeg klikker på en forespørgsel, når den findes, så vises detaljer.
- Givet at jeg ikke er sælger/admin, når jeg forsøger adgang, så bliver jeg afvist.

7. Som sælger ønsker jeg at se detaljer og pris, så jeg kan vurdere forespørgslen.

- Givet at jeg åbner en forespørgsel, når siden vises, så ser jeg specifikationer og kundeoplysninger.
- Givet at systemet beregner pris, når data er gyldige, så vises samlet pris.
- Givet at beregningen fejler, når siden indlæses, så vises en fejlbesked.

8. Som sælger ønsker jeg at godkende en forespørgsel, så kunden kan modtage et tilbud.

- Givet at jeg er på detaljesiden, når jeg godkender forespørgslen, så ændres status til "tilbud sendt".
- Givet at jeg justerer prisen, når jeg sender tilbud, så gemmes den nye pris.
- Givet at forespørgslen godkendes, når det sker, så modtager kunden en mail.

9. Som kunde ønsker jeg at modtage et tilbud, så jeg kan acceptere eller afvise.

- Givet at sælger sender tilbud, når det sker, så modtager jeg en mail.
- Givet at jeg accepterer tilbuddet, når jeg klikker, så sendes jeg til betaling.
- Givet at jeg afviser tilbuddet, når jeg vælger det, så ændres status til afvist.

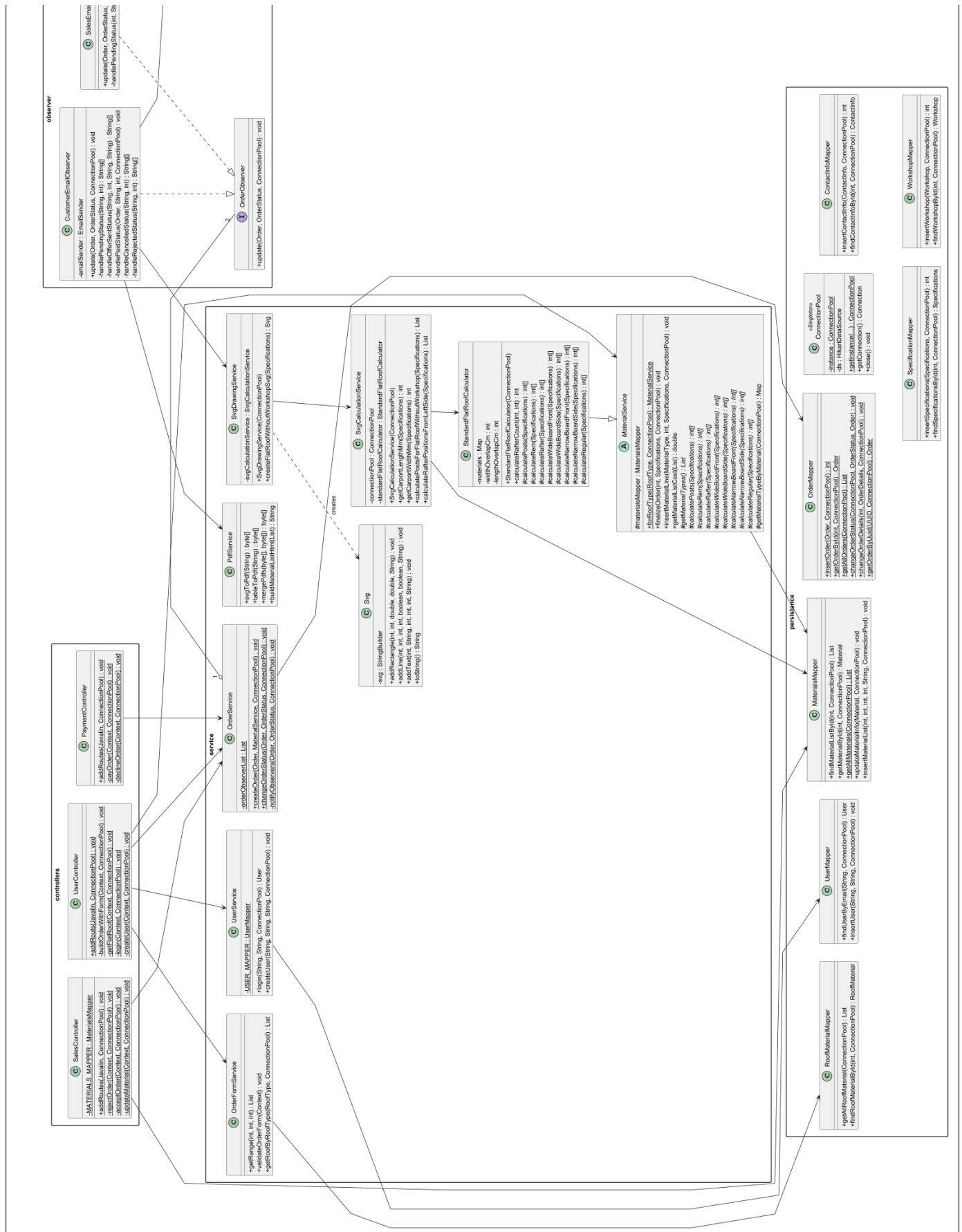
10. Som kunde ønsker jeg en kvittering og stykliste, så jeg kan se hvad jeg har købt.

- Givet at jeg har betalt, når betalingen gennemføres, så ændres status til "betalt".
- Givet at betalingen er gennemført, når det sker, så modtager jeg en kvitteringsmail.
- Givet at jeg har betalt, når jeg tilgår systemet, så får jeg adgang til en stykliste.
- Givet at ordren er betalt, når sælger ser den, så fremgår det at den er klar til behandling.

User story estimat

User story	Estimat
1	Small
2	Large
3	Large
4	Large
5	Small
6	Small
7	Medium
8	Small
9	Medium?
10	Medium

Klassediagram



Kanban Board

	Title	...	Assignees	...	Status	...
1	Frontend - Lav en log ud knap. #43				Done	
2	Backend - Lav funktion på log ud knap. #40				Done	
3	Backend - Vis fejlbeskeder på opret bruger siden. #34				Done	
4	Frontend - Lav startside. #50				Done	
5	Backend - Send mail til kunde og sælger når kunde opretter en forespørgsel, kundens ... #48				Done	
6	Backend - Vis fejlbeskeder på login siden. #35				Done	
7	Frontend - Lav en gem knap som gemmer ændringer når man ændrer et materiales pris. #47				Done	
8	US 7 - Som sælger ønsker jeg at se detaljer og pris, så jeg kan vurdere forespørgslen. #24				Done	
9	US 8 - Som sælger ønsker jeg at godkende en forespørgsel, så kunden kan modtage et ... #25				Done	
10	Frontend - Lav oversigtside over alle forespørgsler. #53				Done	
11	Backend - Lav fejlbesked til materiale-siden (ugyldig pris bl.a.) #42				Done	
12	Frontend - Lav side hvor sælger kan se materialer og priser på dem. #45				Done	
13	Backend - Sørg for at mailen til sælger vedrørende forespørgsel indeholder et link, som... #51				Done	
14	Backend - Vis fejlbesked, hvis linket til forespørgslen er ugyldigt. #56				Done	
15	Database - Lav metoder som henter materialer. #32				Done	
16	Database - Lav metode som ændrer et materiales pris. #33				Done	
17	Database - Opret en ny post i databasen med en given ordres data. #38				Done	
18	Backend - Sørg for, at sælger bliver sendt til login side, hvis ikke sælger er logget ind, n... #55				Done	
19	Frontend - Lave formular/side til at oprette bruger. #39				Done	
20	Backend - Sørg for at en mail sendes til sælger, når der kommer en ny forespørgsel. #49				Done	
21	Frontend - Lav forespørgsels-side, med alle forespørgslens detaljer. #54				Done	
22	Backend - Sørg for at brugere kun har adgang til de sider deres rolle tillader. #37				Done	
23	Frontend - Lave formular/side til at logge ind. #41				Done	
24	Backend - Sørg for at brugeren forbliver logget ind gennem hele session. #36				Done	
25	Frontend - Lav de to sider hvor man kan udfylde data til sin carport. #52				Done	
26	Database - Lav metode til at hente bruger fra databasen. #57				Done	
27	Backend - Lav fejlbesked til de to carport sider. #46				Done	
28	Database - Lav metode til at oprette bruger i database #31				Done	
29	Backend - Lav routing til de forskellige formular-sider. #44				Done	
30	US 6 - Som sælger ønsker jeg at se alle forespørgsler, så jeg kan give tilbud. #23				Done	
1	US 7 - Som sælger ønsker jeg at se detaljer og pris, så jeg kan vurdere forespørgslen. #24				Done	
2	US 5 - Som sælger ønsker jeg at modtage en mail, så jeg kan behandle forespørgsler. #22				Done	
3	US 4 - Som kunde ønsker jeg at vælge carport type og udfylde en formular med mine ø... #21				Done	
4	US 3 - Som sælger ønsker jeg at kunne ændre priser på materialer, så priserne matcher ... #20				Done	
5	US 2 - Som sælger ønsker jeg at kunne logge ind, så jeg får adgang til sælger-funktioner. #19				Done	
6	US 9 - Som kunde ønsker jeg at modtage et tilbud, så jeg kan acceptere eller afvise. #26				Done	
7	US 10 - Som kunde ønsker jeg en kvittering og stykliste, så jeg kan se hvad jeg har købt. #27				Done	
8	US 8 - Som sælger ønsker jeg at godkende en forespørgsel, så kunden kan modtage et ... #25				Done	
9	Database - Lav metode til at hente bruger fra databasen. #57				Done	
10	Database - Lav metode til at oprette bruger i database #31				Done	
11	US 6 - Som sælger ønsker jeg at se alle forespørgsler, så jeg kan give tilbud. #23				Done	
12	US 1 - Som sælger eller kunde ønsker jeg at kunne oprette mig i systemet, så jeg kan få... #18				Done	

Kode eksempler

```
public abstract class MaterialService {

    MaterialsMapper materialsMapper = new MaterialsMapper();

    public static MaterialService forRoofType(RoofType roofType, ConnectionPool connectionPool)
        throws UnsupportedOperationException, DatabaseException {

        return switch (roofType) {
            case FLAT -> new StandardFlatRoofCalculator(connectionPool);
            case RAISED ->
                throw new UnsupportedOperationException("Beregning for rejst tag er ikke implementeret endnu");
        };
    }

    public void finalizeOrder(int orderId, Specifications specifications, ConnectionPool connectionPool) throws DatabaseException {
        List<MaterialType> materialTypes = getMaterialTypes();
        insertAllMaterials(materialTypes, orderId, specifications, connectionPool);
    }

    protected void insertAllMaterials(List<MaterialType> materialTypes, int orderId,
        Specifications specifications, ConnectionPool connectionPool) throws DatabaseException {

        for (MaterialType materialType : materialTypes) {
            insertMaterialLine(materialType, orderId, specifications, connectionPool);
        }
    }
}
```

```

@Test
void calculatePostsShouldAlwaysFollowTheRules() throws DatabaseException {
    StandardFlatRoofCalculator calculator = new StandardFlatRoofCalculator(connectionPool);

    for (int lengthCm = 200; lengthCm <= 2000; lengthCm++) {

        Specifications specs = new Specifications.Builder()
            .lengthCm(lengthCm)
            .widthCm(300)
            .roofType(RoofType.FLAT)
            .roofMaterial(new RoofMaterial.Builder()
                .id(1)
                .name("Test Material")
                .color("Black")
                .price(1000)
                .roofType(RoofType.FLAT)
                .build())
            .build();

        int[] totalPosts = calculator.calculatePosts(specs);

        // RULE 1: Number of posts must be even
        assertTrue(totalPosts[0] % 2 == 0,
            "Length " + lengthCm + " cm gave odd number of posts: " + totalPosts);

        int postsPerSide = totalPosts[0] / 2;
        int roomBetween = lengthCm - 200;
        int gaps = postsPerSide - 1;

        // ONLY check spacing if there are at least 2 posts per side (gaps > 0)
        if (gaps > 0) {
            double spacing = (double) roomBetween / gaps;

            // RULE 2: Max allowed spacing is 300 cm
            assertTrue(spacing <= 300.0,
                "Length " + lengthCm + " cm gave spacing " + spacing + " cm > 300 cm");

            // RULE 3: Spacing must be positive
            assertTrue(spacing > 0,
                "Length " + lengthCm + " cm gave spacing <= 0");
        }
    }
}

```